CSC 4814 & 6814 Efficient AI

Mid-term Exam Review Slides

Dr. Haoxin Wang

Assistant Professor, GSU

https://www.amai-gsu.us/

haoxinwang@gsu.edu

This course draws inspiration and selected materials from Prof. Song Han's MIT course on TinyML and Efficient Deep Learning Computing. Credit is gratefully acknowledged.

Mid-Term Exam

- Mid-term exam: 10/12/2026 (Lectures 1-7)
- 2 separate exam papers: one for undergraduate students, another for graduate students
- Questions: multi-choice, conceptual, short-answer
- You can have a **hand-written** cheat sheet (1-page, single-side, U.S. letter, 8.5 × 11 inches)



Example Question 1

Suppose you have an input of shape **(batch size = 4, input channels = 2)**.

If you use a **fully connected (dense) layer** with 6 output neurons:

(a) What is the weight matrix shape?

(b) How many trainable parameters (weight matrix + biases) does this layer have?

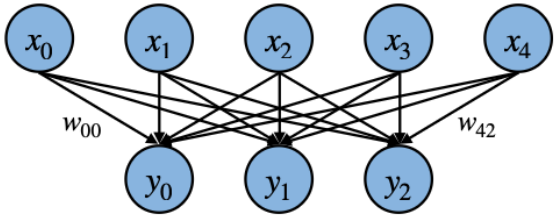


Fully-Connected Layer (Linear layer)

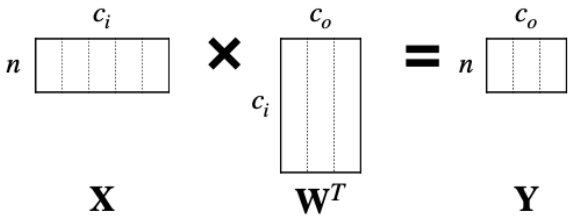
The output neuron is connected to all input neurons.

- **Shape of Tensors:**
  - Input Features $\mathbf{X}$: (n, c_i)
  - Output Features $\mathbf{Y}$: (n, c_o)
  - Weights $\mathbf{W}$: (c_o, c_i)
  - Bias $\mathbf{b}$: $(c_o,)$

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels



$$y_i = \sum_j w_{ij} x_j + b_i$$



Example Question 1- Solution

(a) Weight matrix shape = **(6, 2)**

(b) Trainable parameters = **2x6+6=18**



Example Question 2

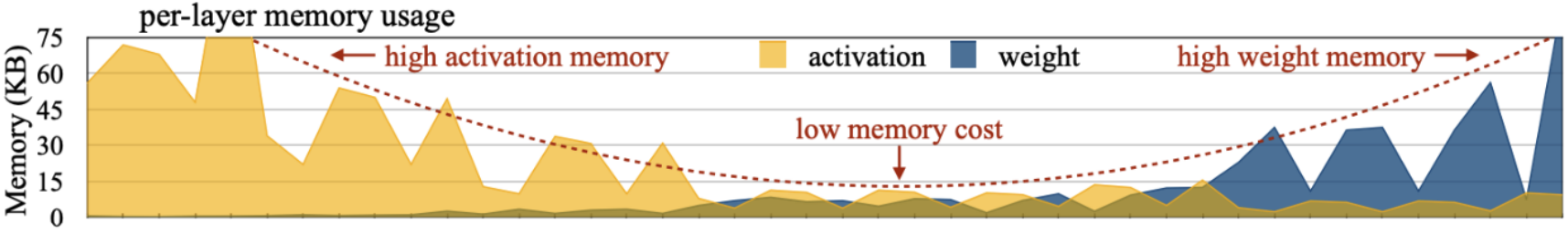
1. Why does **imbalanced memory distribution** happen in CNNs?

2. Suppose you double the input image size (e.g., from 224×224 to 448×448).
   - How does it affect the **#parameters** of the first convolution layer?
   - How does it affect the **#activations** of the first convolution layer?



Number of Activations (#Activations)

Activations and weight memory distribution of CNN



MCUNet: Tiny Deep Learning on IoT Devices [Lin *et al.*, NeurIPS 2020]

Example Question 2 - Solution

1. Early layers: high resolution (large H×W), so activations dominate memory. Later layers: smaller H×W due to downsampling, even if channels increase, memory is much smaller.

2. **#Parameters**: Unchanged (conv kernel size and channels don't depend on input size).
#Activations: Increases roughly 4× (since H and W both double → area scales by factor of 4).



Example Question 3

1. Suppose you are pruning a neural network layer with weight vector: W=[0.05, −0.9, 1.2, 0.02]; which weight would you prune first using Magnitude-based pruning? Why?



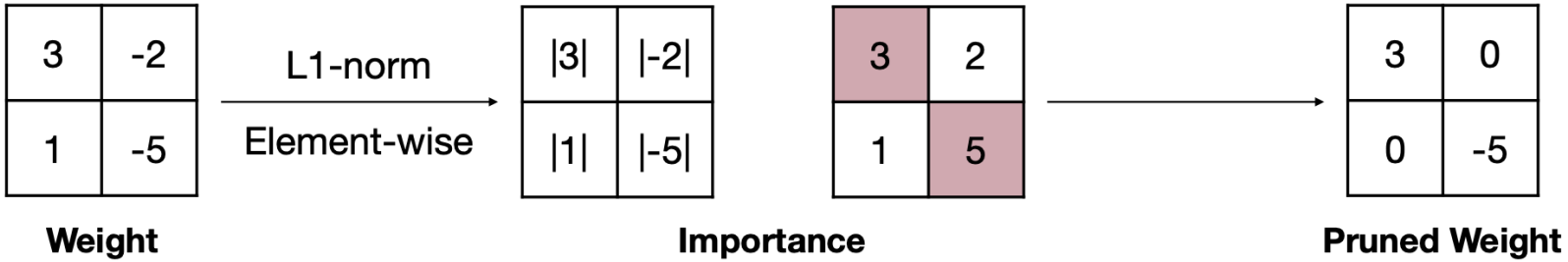
Magnitude-based Pruning

A heuristic pruning criterion

- Magnitude-based pruning considers weights with ***larger absolute values*** are more important than other weights.
  - For element-wise pruning,

$$Importance = |W|$$

- **Example**



Learning Both Weights and Connections for Efficient Neural Network [Han *et al*., NeurIPS 2015]

Example Question 3 - Solution

1. Magnitude-based pruning removes weights with the smallest absolute value first, since they contribute least to the output. Prune first: weight 0.02. It has the smallest magnitude, meaning it has the least influence on the neuron's activation and loss.



Example Question 4

1. Why is finding per-layer pruning ratios challenging, and why does ignoring interactions between layers lead to suboptimal results?
2. Which method formulates pruning as a reinforcement learning problem?
   a) NetAdapt
   b) AMC (AutoML for Model Compression)
   c) Iterative Pruning
   d) Percentage-of-zero pruning

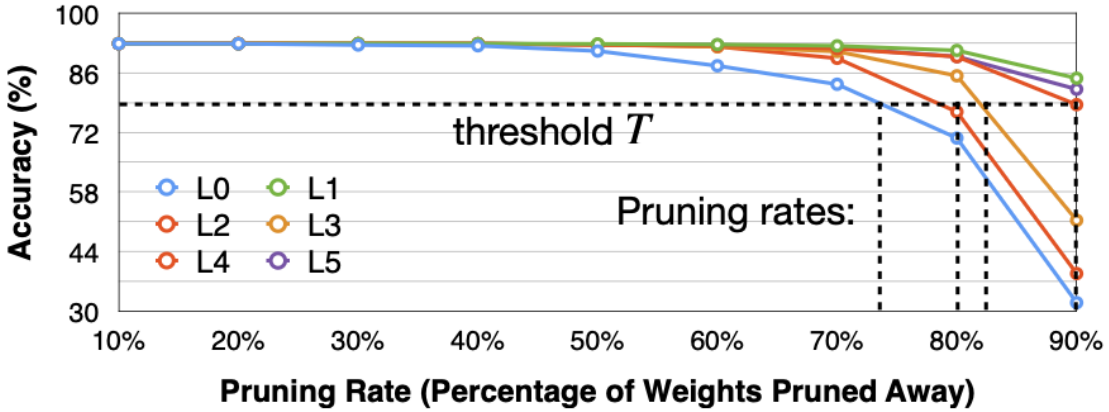


Finding Pruning Ratios

Analyze the sensitivity of each layer

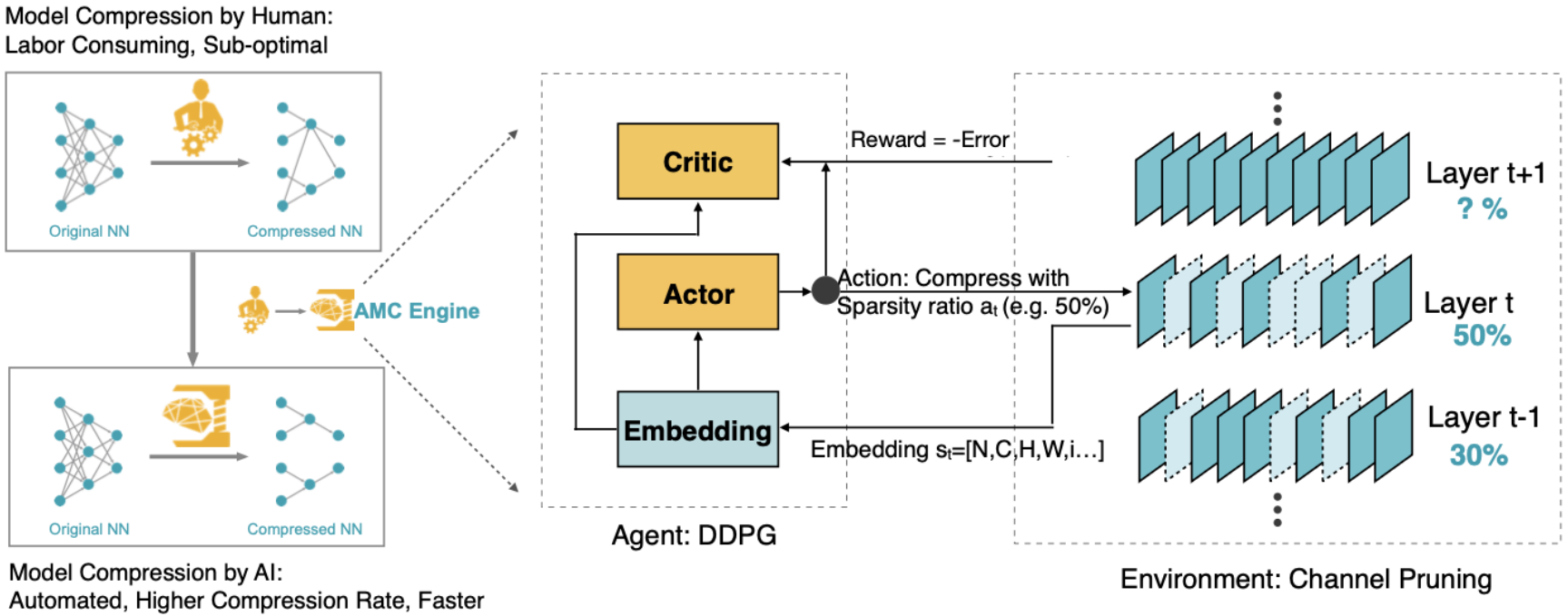
- Is this optimal?
  - Maybe not. We do not consider the interaction between layers.
- Can we go beyond the heuristics?
  - Yes!

AutoML



AMC: AutoML for Model Compression

Pruning as a reinforcement learning problem



AMC: AutoML for Model Compression and Acceleration on Mobile Devices [He *et al.*, ECCV 2018]

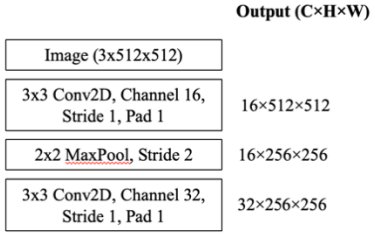
Example Question 4 - Solution

1. Because pruning one layer affects the input distribution of subsequent layers. Ignoring cross-layer interactions leads to overestimation of robustness and poor overall accuracy.
2. b) AMC (AutoML for Model Compression)



Example Question 5

- Given the CNN architecture shown in the figure and input image of size 3x512x512
  - Compute the number of trainable parameters (weights only) for each layer.
  - Compute the number of activations for each layer.
  - Compute the number of MACs for each layer.



Layer	#Parameters
Linear Layer	$c_o \cdot c_i$
Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w$
Grouped Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w / g$
Depthwise Convolution	$c_o \cdot k_h \cdot k_w$

Layer	MACs (batch size n=1)
Linear Layer	$c_o \cdot c_i$
Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w \cdot h_o \cdot w_o$
Grouped Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w \cdot h_o \cdot w_o / g$
Depthwise Convolution	$c_o \cdot k_h \cdot k_w \cdot h_o \cdot w_o$

AlexNet: #Parameters

AlexNet	C × H × W	#Parameters (bias is ignored)
Image (3×224×224)	3×224×224	
11×11 Conv, channel 96, stride 4, pad 2	96×55×55	96×3×11×11 = 24, 848
3×3 MaxPool, stride 2	96×27×27	
5×5 Conv, channel 256, pad 2, groups 2	256×27×27	256×96×5×5 / 2 = 307, 200
3×3 MaxPool, stride 2	256×13×13	
3×3 Conv, channel 384, pad 1	384×13×13	384×256×3×3 = 884, 736
3×3 Conv, channel 384, pad 1, groups 2	384×13×13	384×384×3×3 / 2 = 663, 552
3×3 Conv, channel 256, pad 1, groups 2	256×13×13	256×[illegible]×3×3 / 2 = 442, 368
3×3 MaxPool, stride 2	256×6×6	
Linear, channel 4096	4096	4096×(256×6×6) = 37, 748, 736
Linear, channel 4096	4096	4096×4096 = 16, 777, 216
Linear, channel 1000	1000	1000×4096 = 4, 096, 000

Layer	#Parameters
Linear Layer	$c_o \cdot c_i$
Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w$
Grouped Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w / g$
Depthwise Convolution	$c_o \cdot k_h \cdot k_w$

61M in total

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky *et al.*, NeurIPS 2012]

	Output (C×H×W)
Image (3x512x512)	
3x3 Conv2D, Channel 16, Stride 1, Pad 1	16×512×512
2x2 MaxPool, Stride 2	16×256×256
3x3 Conv2D, Channel 32, Stride 1, Pad 1	32×256×256
2x2 MaxPool, Stride 2	32×128×128
3x3 Conv2D, Channel 64, Stride 1, Pad 1	64×128×128
2x2 MaxPool, Stride 2	64×64×64
Linear, channel 100	100

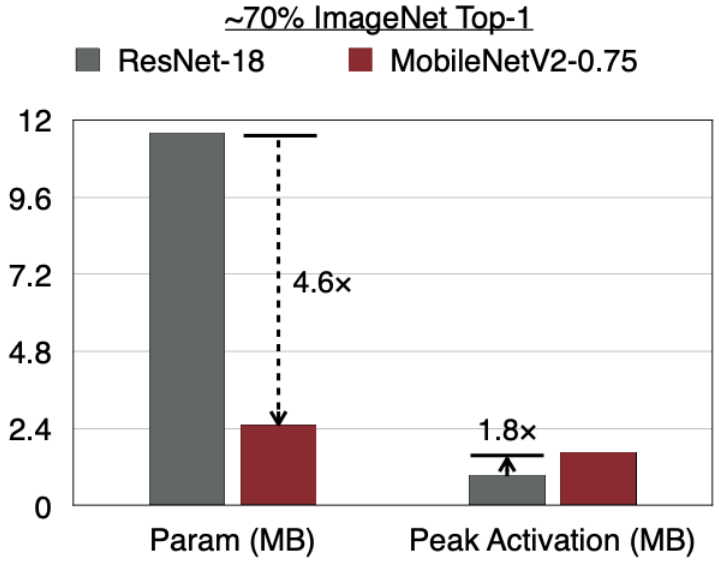


AlexNet	C × H × W
Image (3×224×224)	3×224×224
11×11 Conv, channel 96, stride 4, pad 2	96×55×55
3×3 MaxPool, stride 2	96×27×27
5×5 Conv, channel 256, pad 2, groups 2	256×27×27
3×3 MaxPool, stride 2	256×13×13
3×3 Conv, channel 384, pad 1	384×13×13
3×3 Conv, channel 384, pad 1, groups 2	384×13×13
3×3 Conv, channel 256, pad 1, groups 2	256×13×13
3×3 MaxPool, stride 2	256×6×6
Linear, channel 4096	4096
Linear, channel 4096	4096
Linear, channel 1000	1000

ImageNet Classification with De

Number of Activations (#Activations)

#Activations is the memory bottleneck in CNN inference, not #parameters



*** All parameters and activations are Integer numbers (8 bits).**

MCUNet: Tiny Deep Learning on IoT Devices [Lin *et al.*, NeurIPS 2020]

AlexNet: #Activations



AlexNet	C × H × W	
Image (3×224×224)	3×224×224	=150,528
11×11 Conv, channel 96, stride 4, pad 2	96×55×55	=290,400
3×3 MaxPool, stride 2	96×27×27	=69,984
5×5 Conv, channel 256, pad 2, groups 2	256×27×27	=186,624
3×3 MaxPool, stride 2	256×13×13	=43,264
3×3 Conv, channel 384, pad 1	384×13×13	=64,896
3×3 Conv, channel 384, pad 1, groups 2	384×13×13	=64,896
3×3 Conv, channel 256, pad 1, groups 2	256×13×13	=43,264
3×3 MaxPool, stride 2	256×6×6	=9,216
Linear, channel 4096	4096	=4,096
Linear, channel 4096	4096	=4,096
Linear, channel 1000	1000	=1,000

Total #Activation: 932,264

Peak #Activation:

≈ #input activation + #output activation

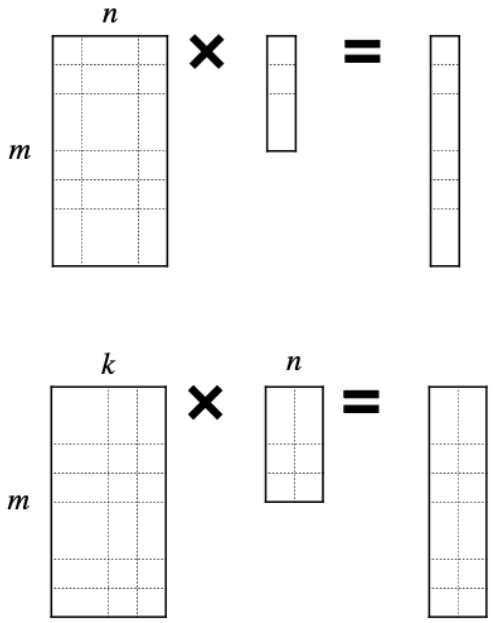
= 150,528 + 290,400 = 440,928

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky *et al.*, NeurIPS 2012]

Number of Multiply-Accumulate Operations

MAC

- Multiply-Accumulate operation (MAC)
  - $a \leftarrow a + b \cdot c$
- Matrix-Vector Multiplication (MV)
  - $MACs = m \cdot n$
- General Matrix-Matrix Multiplication (GEMM)
  - $MACs = m \cdot n \cdot k$



AlexNet: #MACs



AlexNet	C × H × W	MACs
Image (3×224×224)	3×224×224	
11×11 Conv, channel 96, stride 4, pad 2	96×55×55	96×3×11×11×55×55 = 105,415,200
3×3 MaxPool, stride 2	96×27×27	
5×5 Conv, channel 256, pad 2, groups 2	256×27×27	256×96×5×5×27×27 / 2 = 223,948,800
3×3 MaxPool, stride 2	256×13×13	
3×3 Conv, channel 384, pad 1	384×13×13	384×256×3×3×13×13 = 149,520,384
3×3 Conv, channel 384, pad 1, groups 2	384×13×13	384×384×3×3×13×13 / 2 = 112,140,288
3×3 Conv, channel 256, pad 1, groups 2	256×13×13	256×[illegible]×3×13×13 / 2 = 74,[illegible]
3×3 MaxPool, stride 2	256×6×6	
Linear, channel 4096	4096	4096×(256×6×6) = 37,748,736
Linear, channel 4096	4096	4096×4096 = 16,777,216
Linear, channel 1000	1000	1000×4096 = 4,096,000

Layer	MACs (batch size n=1)
Linear Layer	$c_o \cdot c_i$
Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w \cdot h_o \cdot w_o$
Grouped Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w \cdot h_o \cdot w_o / g$
Depthwise Convolution	$c_o \cdot k_h \cdot k_w \cdot h_o \cdot w_o$

724M in total !

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky *et al.*, NeurIPS 2012]

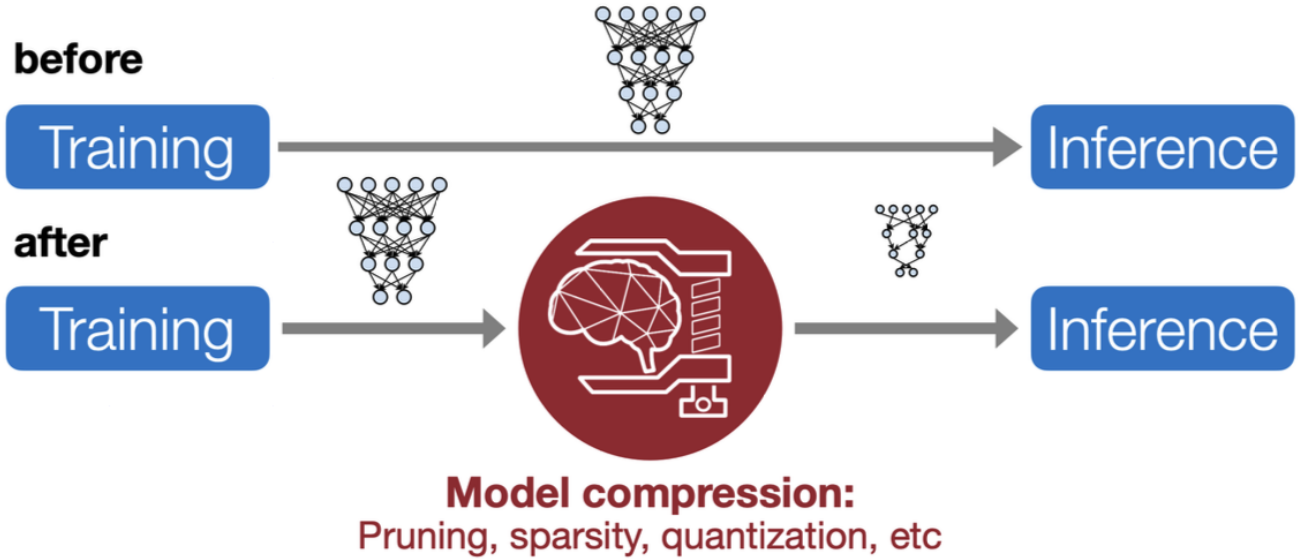
Example Question 5 - Solution

Layer	Input	Output	#Params	#Activations	#MACs
Conv1 3→16, 3×3 s1 p1	3×512×512	16×512×512	432	4,194,304	113,246,208
MaxPool 2×2 s2	16×512×512	16×256×256	0	1,048,576	0
Conv2 16→32, 3×3 s1 p1	16×256×256	32×256×256	4,608	2,097,152	301,989,888



Model Compression and Efficient AI

- **Bridges the Gap between the Supply and Demand of AI Computing**



This course draws inspiration and selected materials from Prof. Song Han's MIT course on TinyML and Efficient Deep Learning Computing. Credit is gratefully acknowledged.

Trade-off Between Efficiency and Accuracy

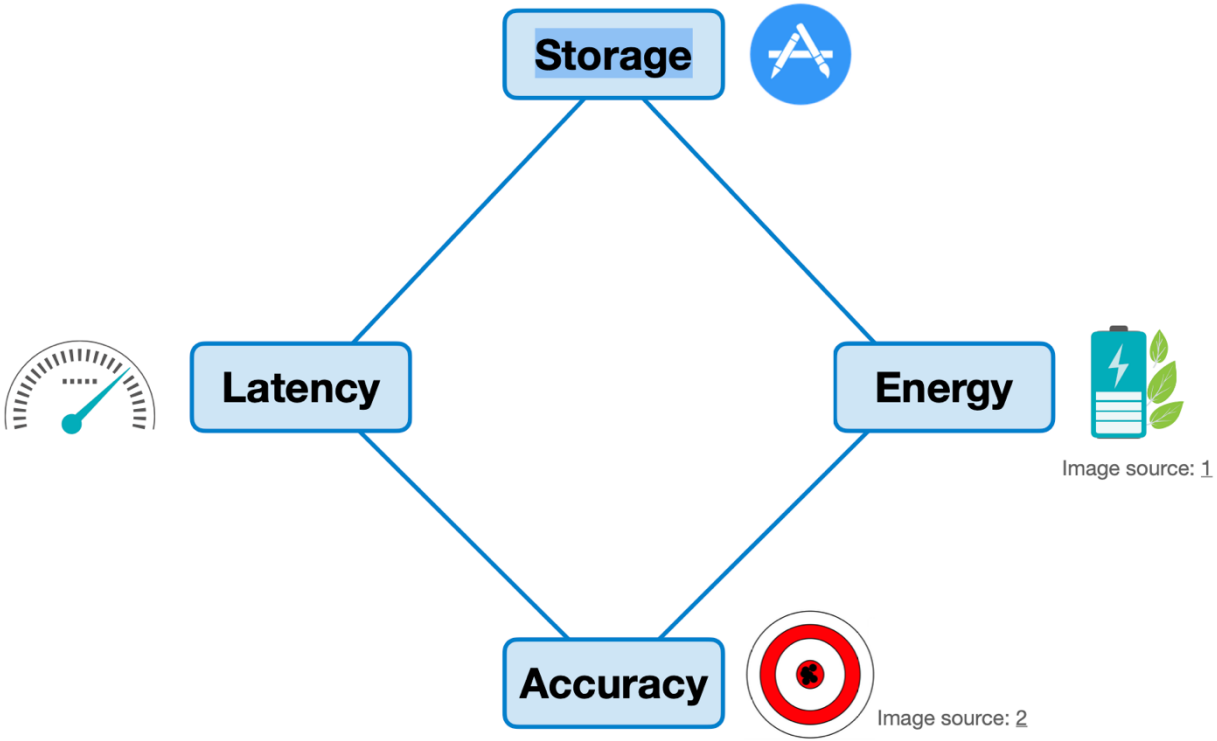
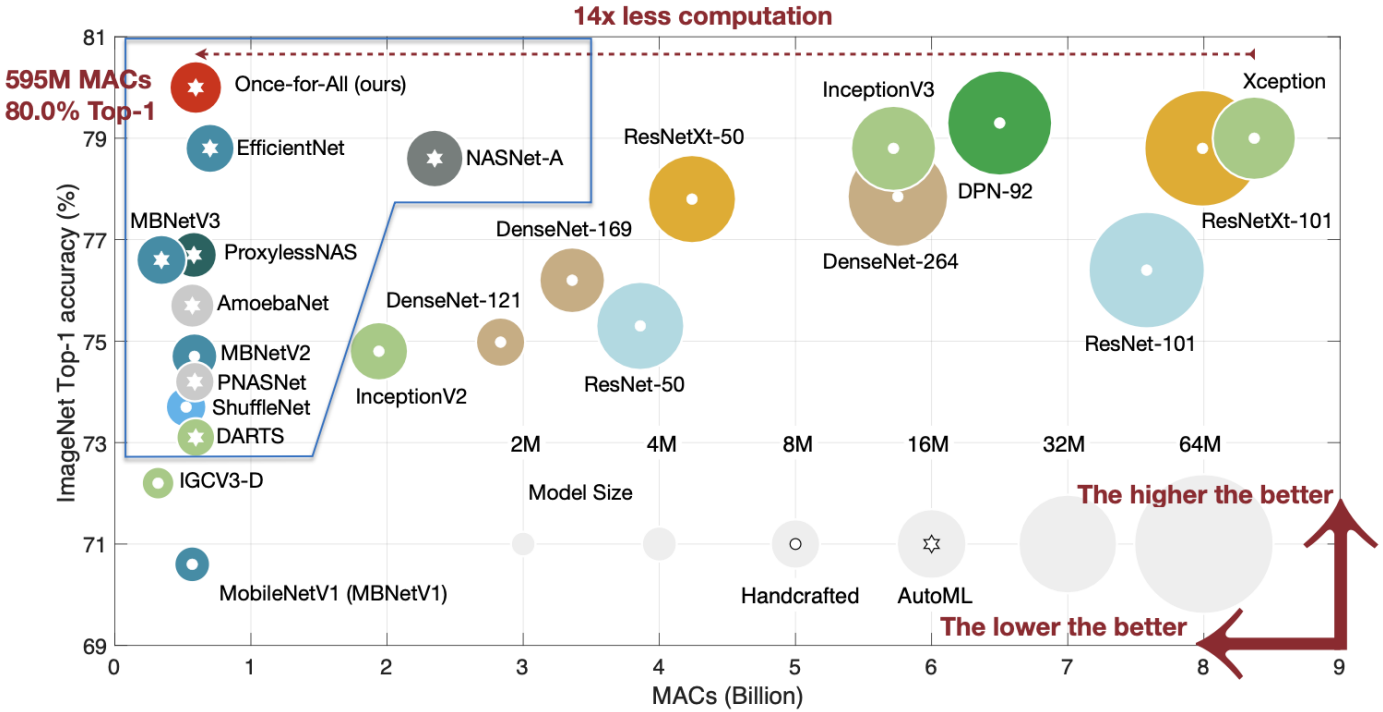


Image source: 1

Image source: 2

Efficient Image Classification

- **Neural architecture search (NAS) reduces the computational cost**



Once-for-All: Train One Network and Specialize it for Efficient Deployment [Cai *et al.*, ICLR 2020]

Large Language Models

Large language models show emergent behaviors: zero/few learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  cheese =>                           ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  sea otter => loutre de mer          ← examples
3  peppermint => menthe poivrée        ←
4  plush girafe => girafe peluche      ←
5  cheese =>                           ← prompt
```



Large Language Models

Large language models show emergent behaviors: chain-of-thought

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ✗

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

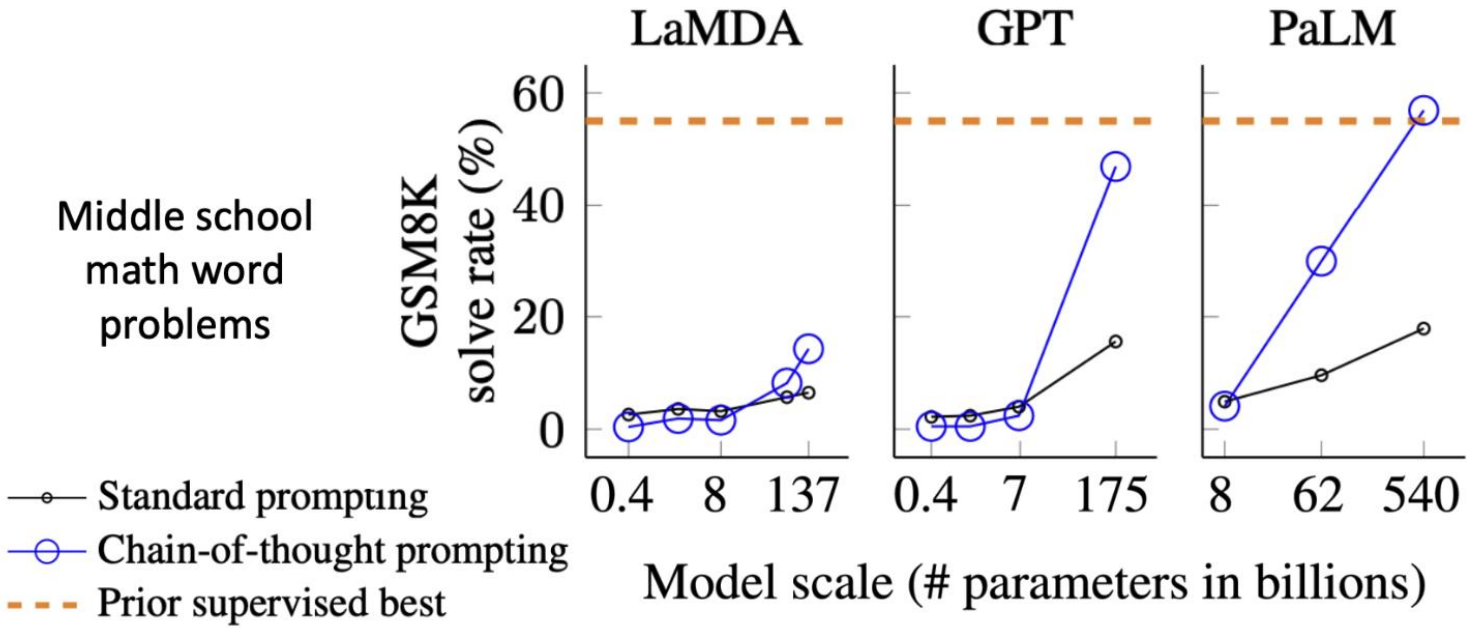
Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

Wei et al., Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, 2022

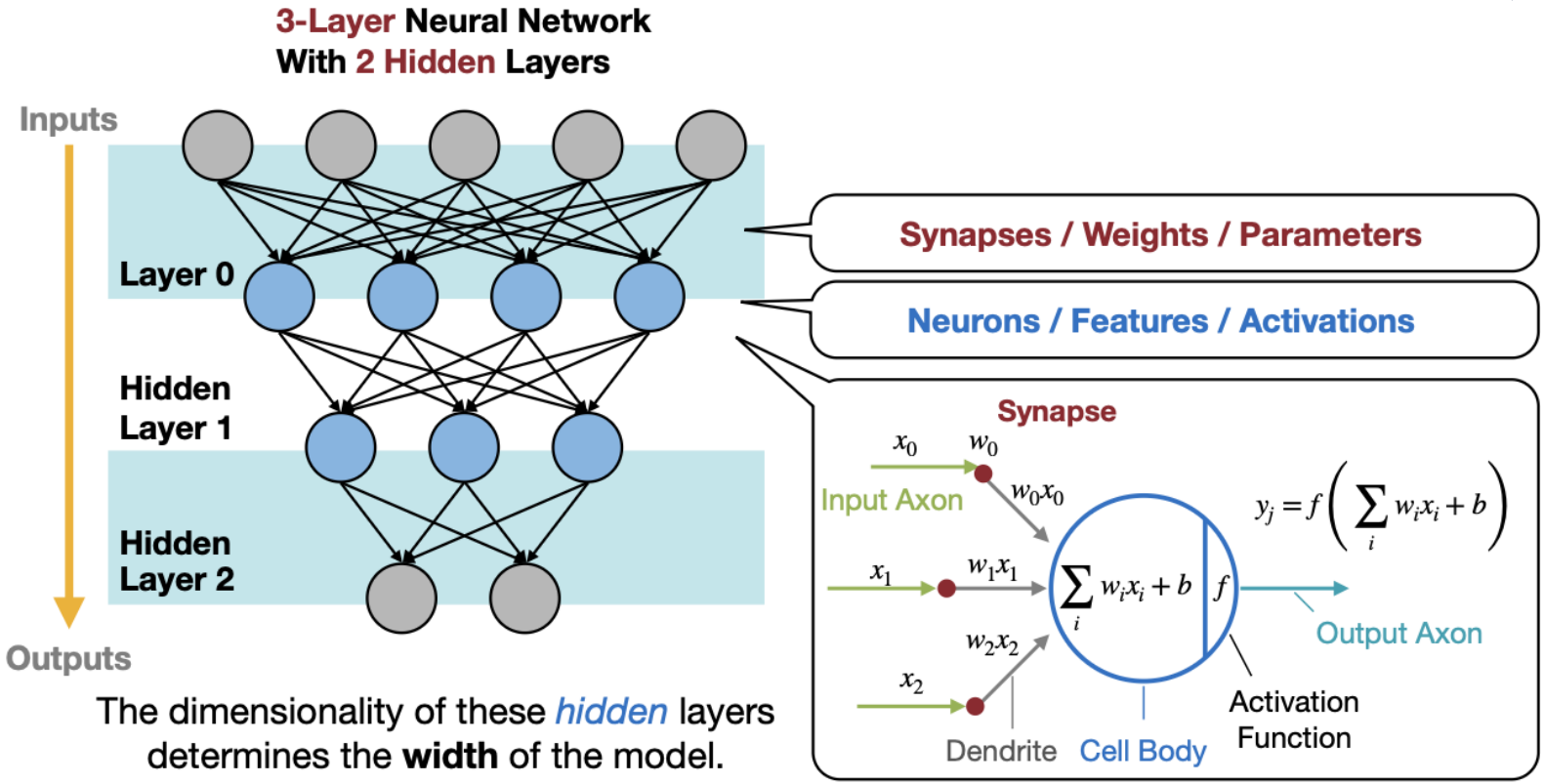
Large Language Models

But it comes at the cost of large model size



Wei et al., Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, 2022

Deep Neural Network



The dimensionality of these *hidden* layers determines the **width** of the model.

Popular Neural Network Layers

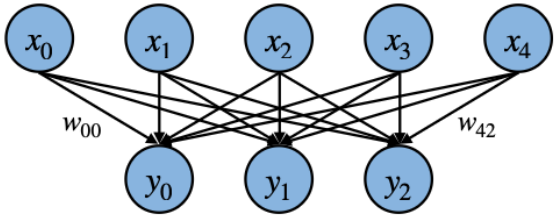


Fully-Connected Layer (Linear layer)

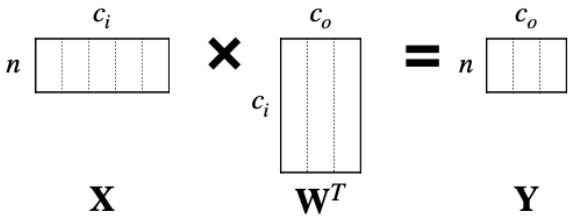
The output neuron is connected to all input neurons.

- **Shape of Tensors:**
  - Input Features $\mathbf{X}$: (n, c_i)
  - Output Features $\mathbf{Y}$: (n, c_o)
  - Weights $\mathbf{W}$: (c_o, c_i)
  - Bias $\mathbf{b}$: $(c_o,)$

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels



$$y_i = \sum_j w_{ij} x_j + b_i$$

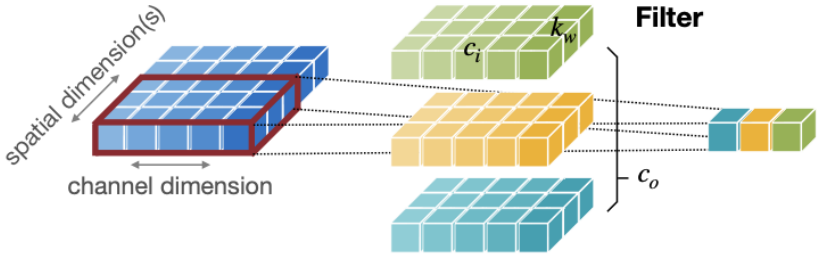


Convolution Layer

The output neuron is connected to input neurons in the receptive field.

- **Shape of Tensors:** **1D Conv**
  - Input Features $\mathbf{X}$: ~~(n, c_i)~~ (n, c_i, w_i)
  - Output Features $\mathbf{Y}$: ~~(n, c_o)~~ (n, c_o, w_o)
  - Weights $\mathbf{W}$: ~~(c_o, c_i)~~
  - Bias $\mathbf{b}$: $(c_o,)$

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
w_i, w_o	Input/Output Width
k_w	Kernel Width



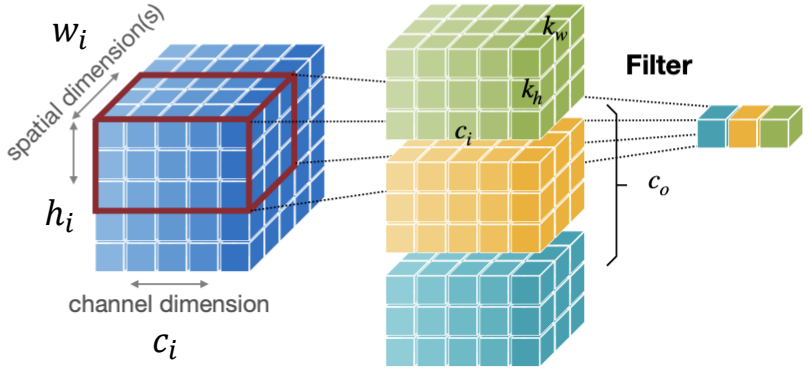
Convolution Layer

The output neuron is connected to input neurons in the receptive field.

- **Shape of Tensors:**

	1D Conv	2D Conv
Input Features $\mathbf{X}$: ~~(n, c_i)~~	(n, c_i, w_i)	(n, c_i, h_i, w_i)
Output Features $\mathbf{Y}$: ~~(n, c_o)~~	(n, c_o, w_o)	(n, c_o, h_o, w_o)
Weights $\mathbf{W}$: ~~(c_o, c_i)~~	(c_o, c_i, k_w)	(c_o, c_i, k_h, k_w)
Bias $\mathbf{b}$: $(c_o,)$		

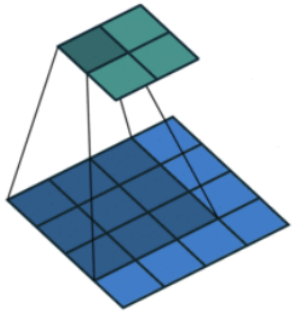
Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h	Kernel Height
k_w	Kernel Width

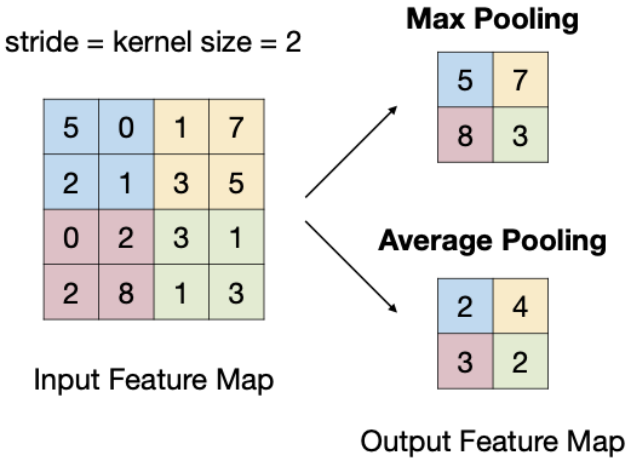


Pooling Layer

Downsample the feature map to a smaller size

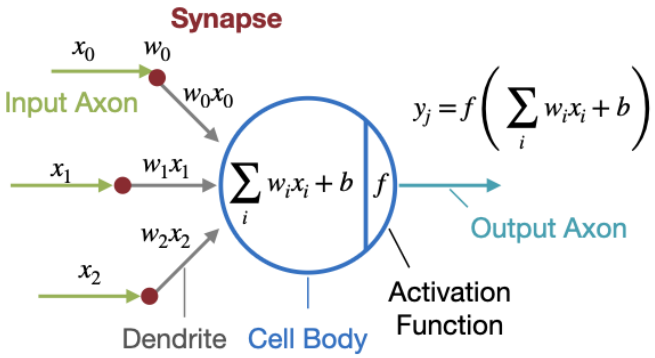
- The output neuron pools the features in the receptive field, similar to convolution.
  - Usually, the stride is the same as the kernel size: $s = k$
- Pooling operates over each channel independently.
  - No learnable parameters

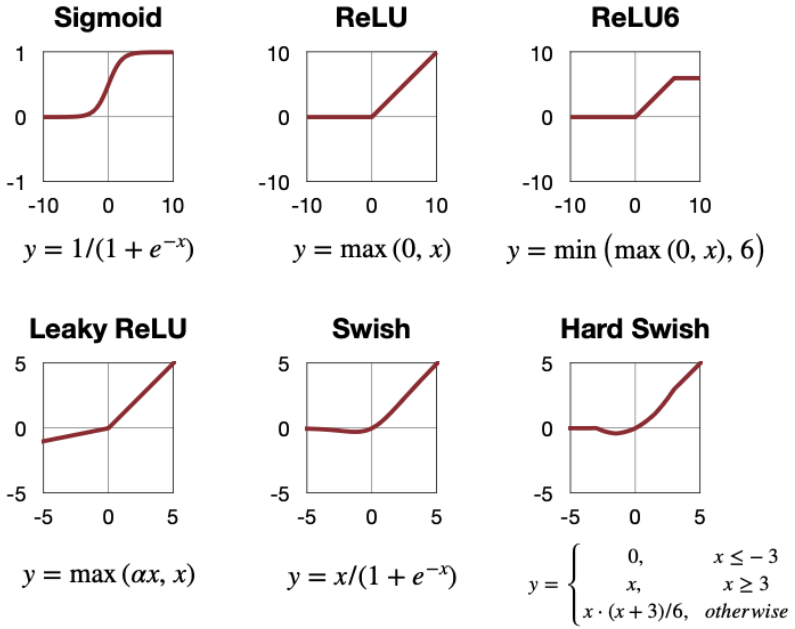




Activation Function

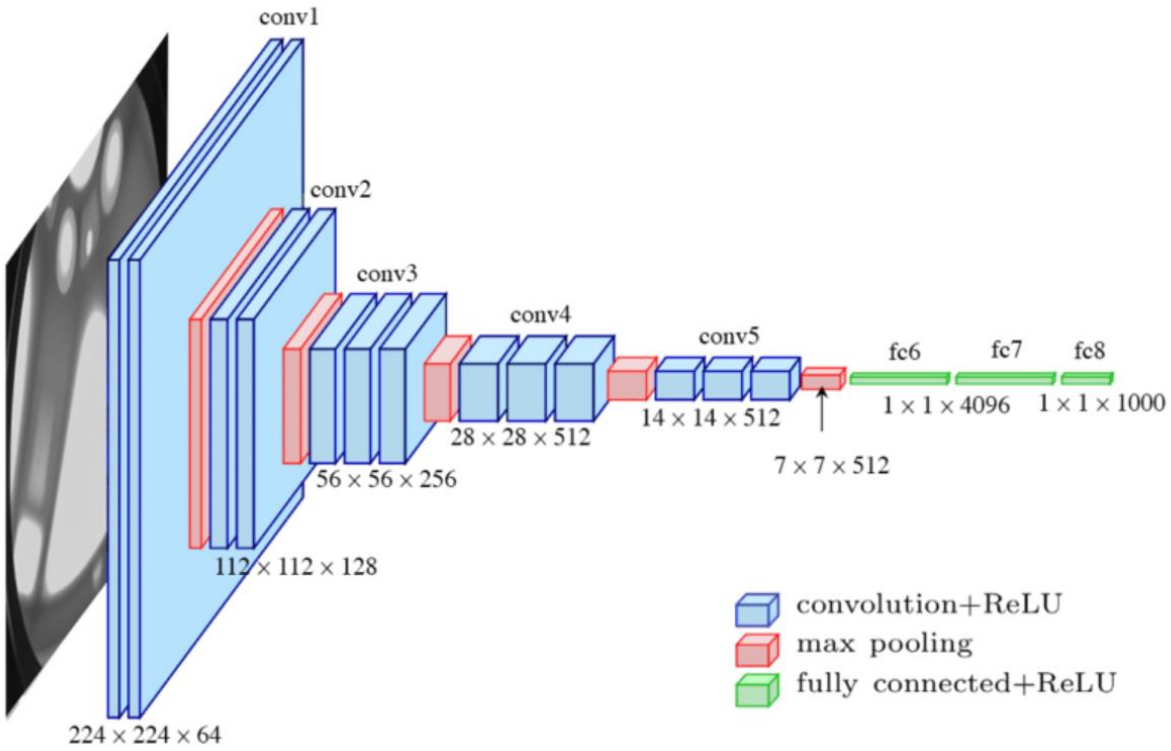
Activation functions are typically non-linear functions





Other Activation Functions: Tanh, GELU, ELU, Mish…

CNN Architecture Example

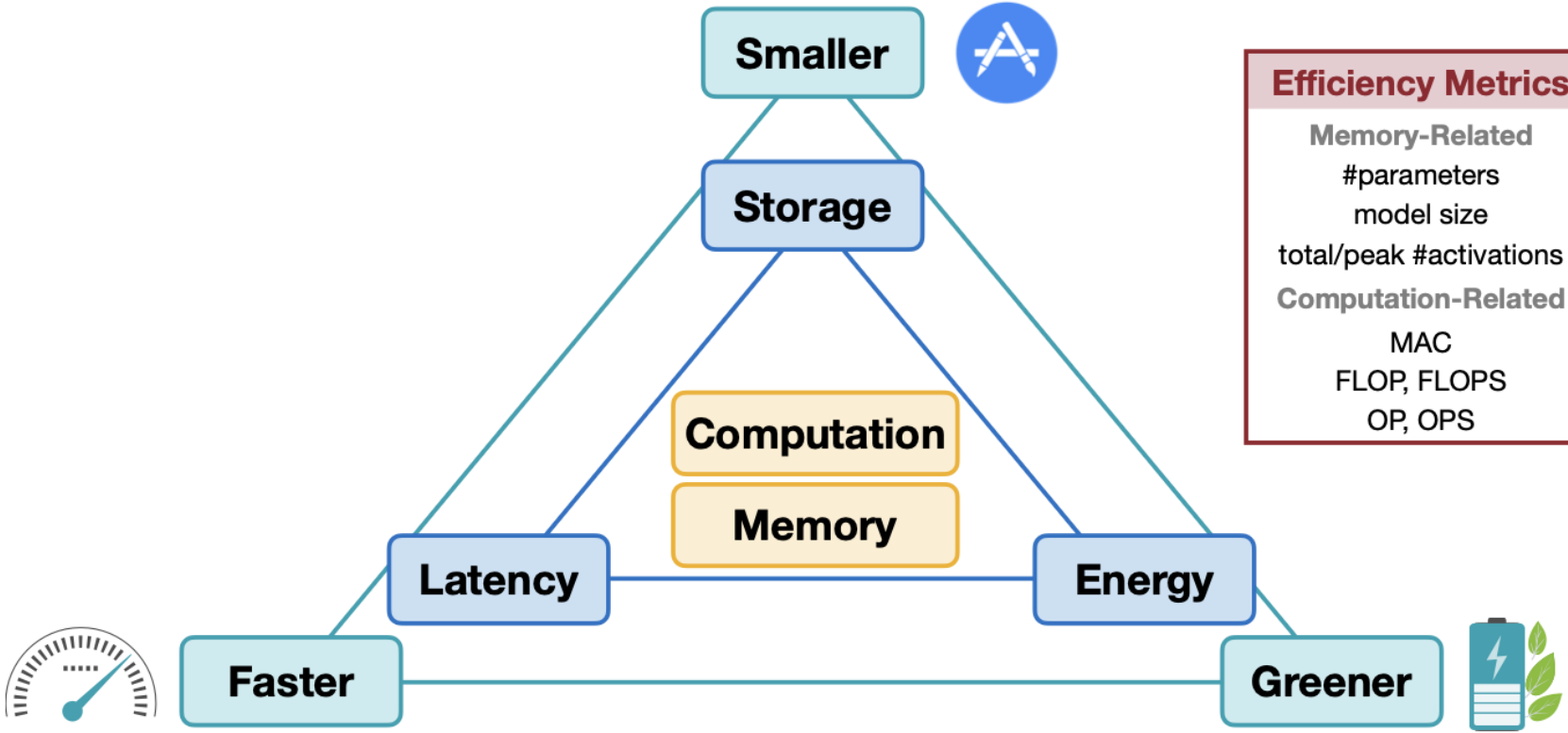


Efficiency Metrics

How should we measure the efficiency of neural networks?



Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

FLOP, FLOPS

OP, OPS

Latency

Measures the delay for a specific task



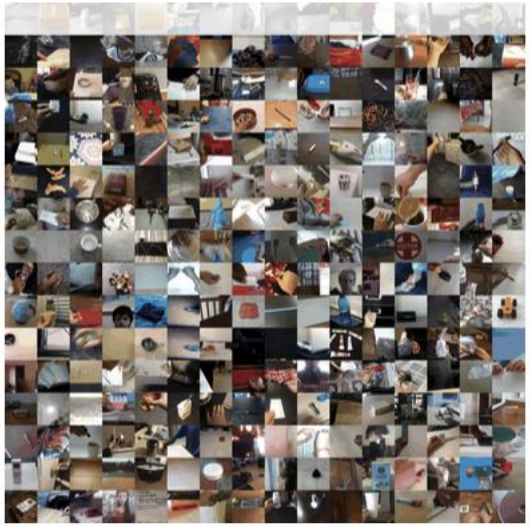
SegFormer
638ms, 82.4mIoU



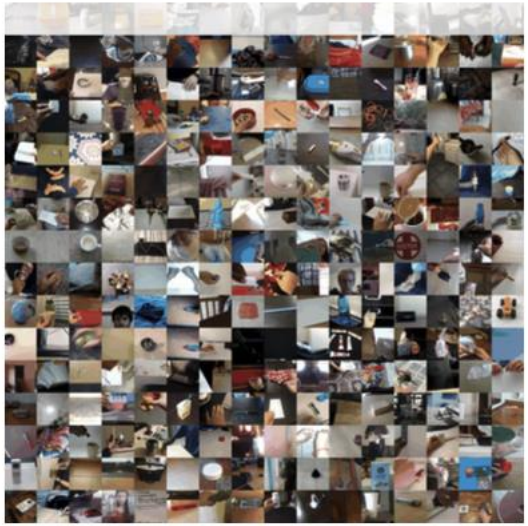
EfficientViT
46ms, 82.7mIoU

Throughput

Measures the rate at which data is processed



Low Throughput = 6.1 video/s



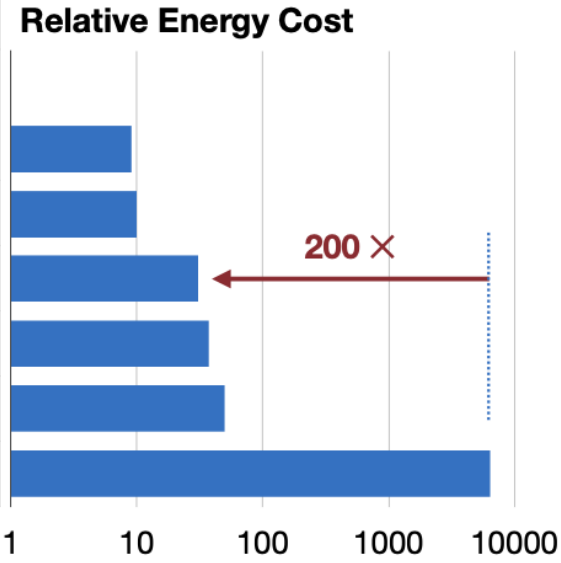
High Throughput = 77.4 video/s

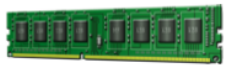
Energy Consumption

Data movement → more memory reference → more energy

Operation	Energy [pJ]
32 bit int ADD	0.1
32 bit float ADD	0.9
32 bit Register File	1
32 bit int MULT	3.1
32 bit float MULT	3.7
32 bit SRAM Cache	5
32 bit DRAM Memory	640

Rough Energy Cost For Various Operations in 45nm 0.9V

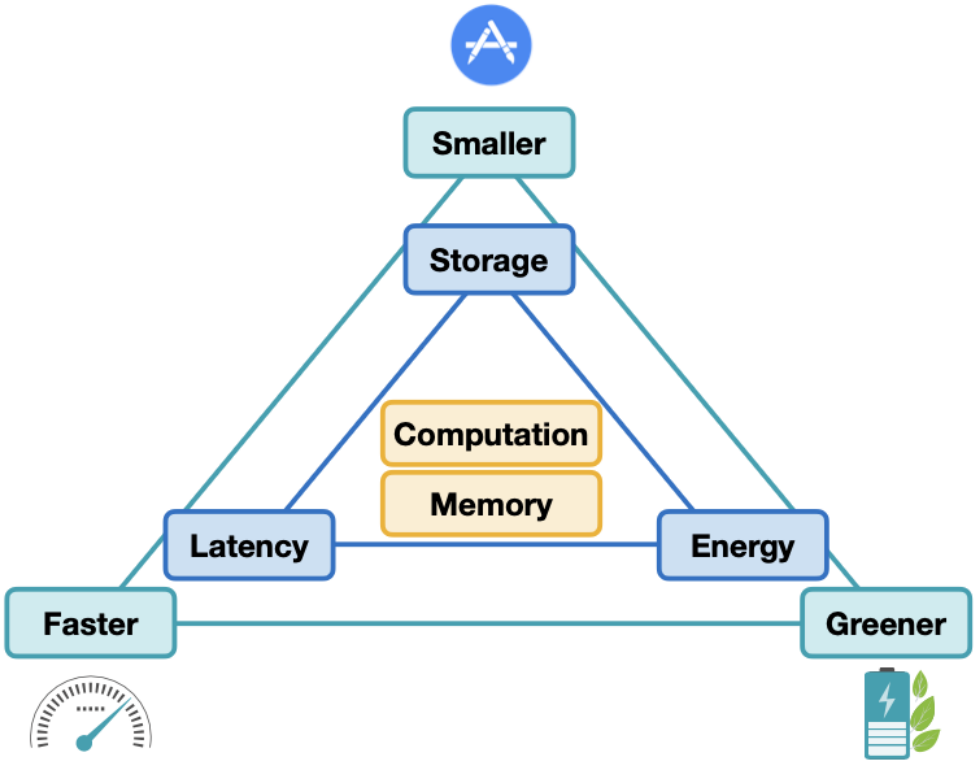


1  = 200 ×+

This image is in the public domain

Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]

Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

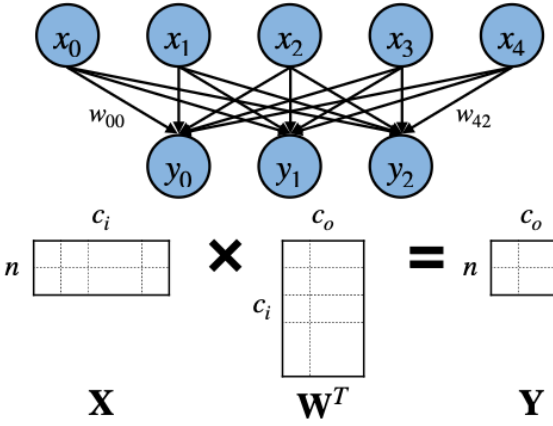
FLOP, FLOPS

OP, OPS

Number of Parameters (# Parameters)

#Parameters is the parameter (synapse/weight) count of the given neural network, *i.e.*, the number of elements in the weight tensors.

Layer	#Parameters (bias is ignored)
Linear Layer	$c_o \cdot c_i$
Convolution	
Grouped Convolution	
Depthwise Convolution	



Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h, k_w	Kernel Height/Width
g	Groups

Number of Parameters (# Parameters)

#Parameters is the parameter (synapse/weight) count of the given neural network, *i.e.*, the number of elements in the weight tensors.

Layer	#Parameters (bias is ignored)
Linear Layer	$c_o \cdot c_i$
Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w$
Grouped Convolution	
Depthwise Convolution	

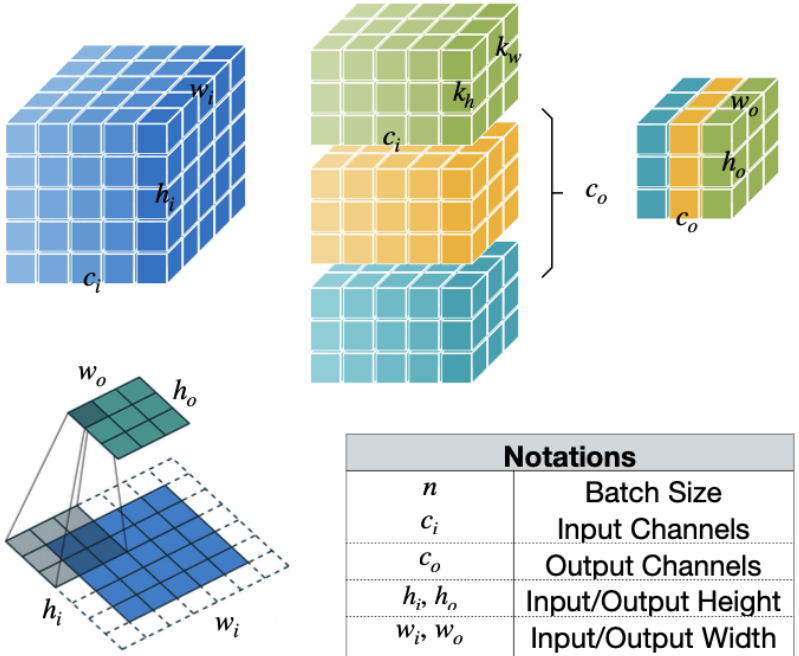


Image source: 1

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h, k_w	Kernel Height/Width
g	Groups

AlexNet: #Parameters

AlexNet	C × H × W	#Parameters (bias is ignored)
Image (3×224×224)	3×224×224	
11×11 Conv, channel 96, stride 4, pad 2	96×55×55	96×3×11×11 = 24, 848
3×3 MaxPool, stride 2	96×27×27	
5×5 Conv, channel 256, pad 2, groups 2	256×27×27	256×96×5×5 / 2 = 307, 200
3×3 MaxPool, stride 2	256×13×13	
3×3 Conv, channel 384, pad 1	384×13×13	384×256×3×3 = 884, 736
3×3 Conv, channel 384, pad 1, groups 2	384×13×13	384×384×3×3 / 2 = 663, 552
3×3 Conv, channel 256, pad 1, groups 2	256×13×13	256×[illegible]×3×3 / 2 = 442, 368
3×3 MaxPool, stride 2	256×6×6	
Linear, channel 4096	4096	4096×(256×6×6) = 37, 748, 736
Linear, channel 4096	4096	4096×4096 = 16, 777, 216
Linear, channel 1000	1000	1000×4096 = 4, 096, 000

Layer	#Parameters
Linear Layer	$c_o \cdot c_i$
Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w$
Grouped Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w / g$
Depthwise Convolution	$c_o \cdot k_h \cdot k_w$

61M in total

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky *et al.*, NeurIPS 2012]

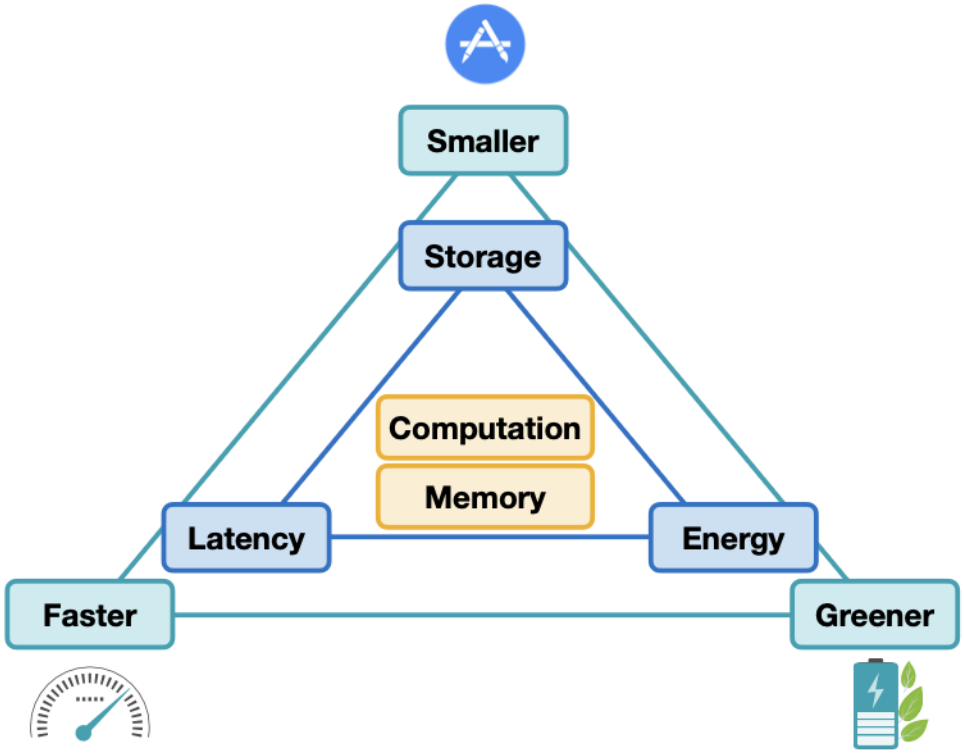
	Output (C×H×W)
Image (3x512x512)	
3x3 Conv2D, Channel 16, Stride 1, Pad 1	16×512×512
2x2 MaxPool, Stride 2	16×256×256
3x3 Conv2D, Channel 32, Stride 1, Pad 1	32×256×256
2x2 MaxPool, Stride 2	32×128×128
3x3 Conv2D, Channel 64, Stride 1, Pad 1	64×128×128
2x2 MaxPool, Stride 2	64×64×64
Linear, channel 100	100



AlexNet	C × H × W
Image (3×224×224)	3×224×224
11×11 Conv, channel 96, stride 4, pad 2	96×55×55
3×3 MaxPool, stride 2	96×27×27
5×5 Conv, channel 256, pad 2, groups 2	256×27×27
3×3 MaxPool, stride 2	256×13×13
3×3 Conv, channel 384, pad 1	384×13×13
3×3 Conv, channel 384, pad 1, groups 2	384×13×13
3×3 Conv, channel 256, pad 1, groups 2	256×13×13
3×3 MaxPool, stride 2	256×6×6
Linear, channel 4096	4096
Linear, channel 4096	4096
Linear, channel 1000	1000

ImageNet Classification with De

Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

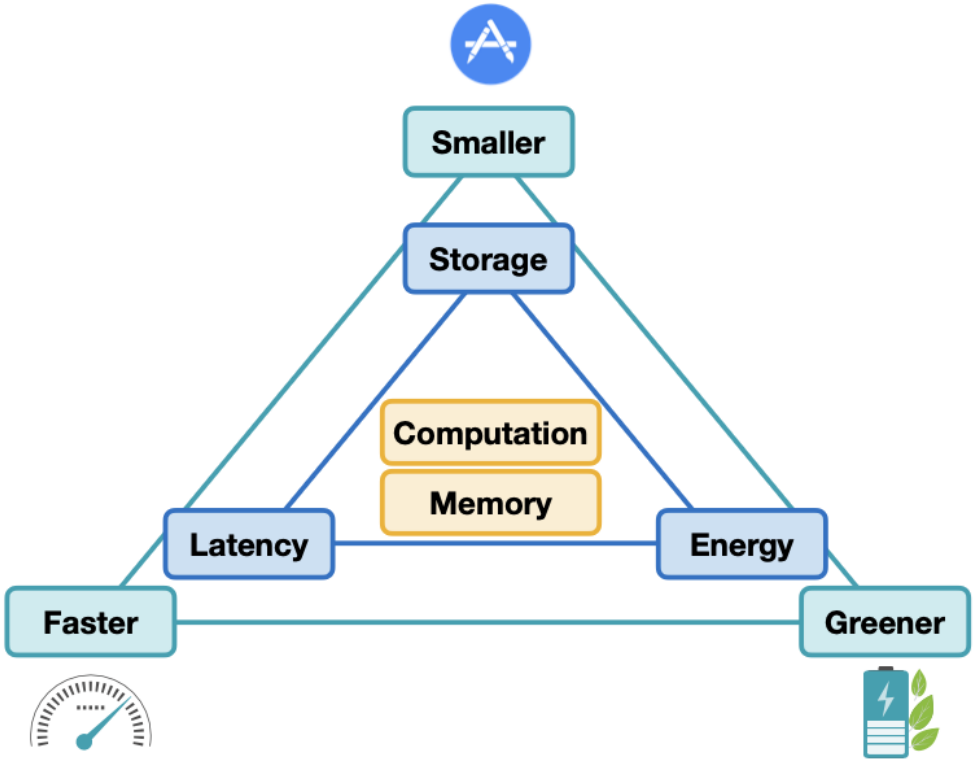
FLOP, FLOPS

OP, OPS

Model Size

- Model size measures the storage for the weights of the given neural network.
  - The common units for model size are: MB (megabyte), KB (kilobyte), bits.
- In general, if the whole neural network uses the same data type (*e.g.*, floating-point),
  - $Model\ \ Size = \#Parameters \cdot Bit\ Width$
  - Example: AlexNet has 61M parameters.
    - If all weights are stored with 32-bit numbers, total storage will be about
      - 61M × 4 Bytes (32 bits) = 244 MB (244×10^6 Bytes)
    - If all weights are stored with 8-bit numbers, total storage will be about
      - 61M × 1 Byte (8 bits) = 61 MB

Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

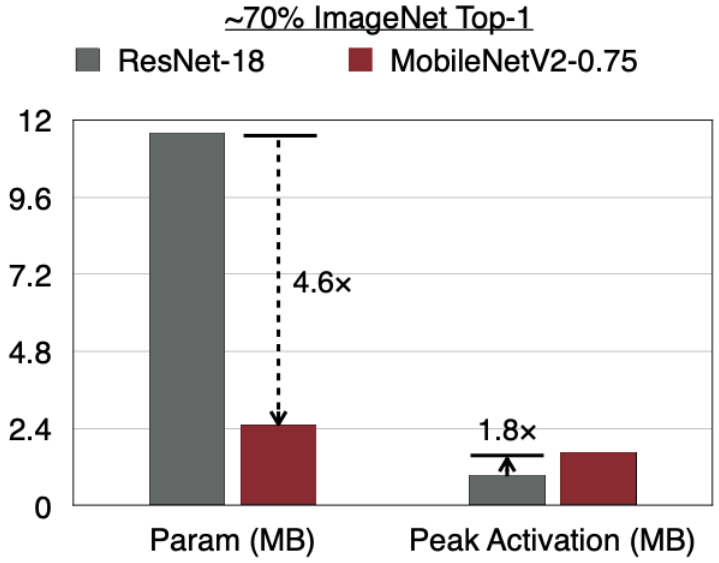
MAC

FLOP, FLOPS

OP, OPS

Number of Activations (#Activations)

#Activations is the memory bottleneck in CNN inference, not #parameters



*** All parameters and activations are Integer numbers (8 bits).**

MCUNet: Tiny Deep Learning on IoT Devices [Lin *et al.*, NeurIPS 2020]

AlexNet: #Activations



AlexNet	C × H × W	
Image (3×224×224)	3×224×224	=150,528
11×11 Conv, channel 96, stride 4, pad 2	96×55×55	=290,400
3×3 MaxPool, stride 2	96×27×27	=69,984
5×5 Conv, channel 256, pad 2, groups 2	256×27×27	=186,624
3×3 MaxPool, stride 2	256×13×13	=43,264
3×3 Conv, channel 384, pad 1	384×13×13	=64,896
3×3 Conv, channel 384, pad 1, groups 2	384×13×13	=64,896
3×3 Conv, channel 256, pad 1, groups 2	256×13×13	=43,264
3×3 MaxPool, stride 2	256×6×6	=9,216
Linear, channel 4096	4096	=4,096
Linear, channel 4096	4096	=4,096
Linear, channel 1000	1000	=1,000

Total #Activation: 932,264

Peak #Activation:

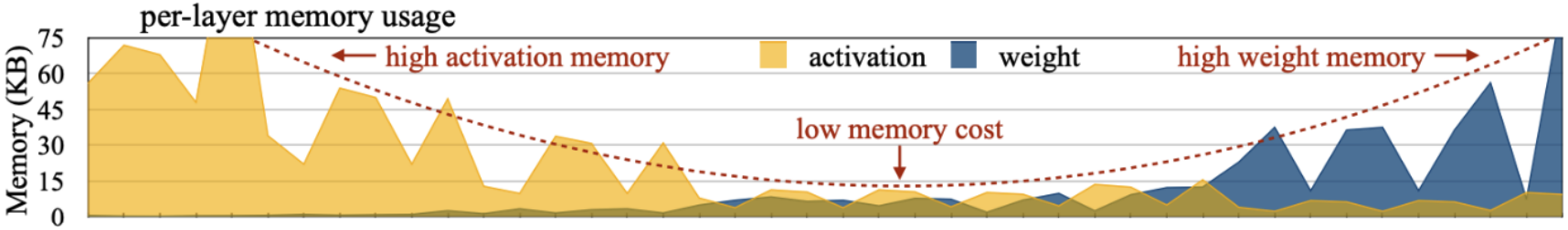
≈ #input activation + #output activation

= 150,528 + 290,400 = 440,928

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky *et al.*, NeurIPS 2012]

Number of Activations (#Activations)

Activations and weight memory distribution of CNN

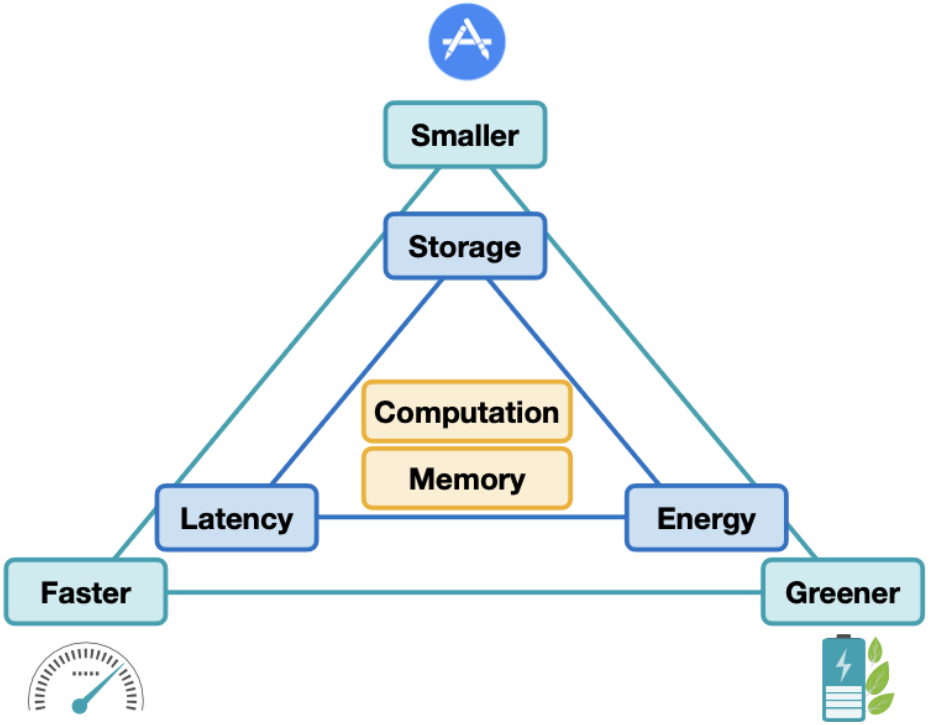


Why does imbalanced memory distribution happen in CNNs?

Early layers operate on large spatial maps (H×W), so activation tensors are huge and dominate memory. As the network downsamples, H and W shrink; later layers have relatively small activations, and parameters (filters) become a larger share. Thus memory distribution is imbalanced across depth.

MCUNet: Tiny Deep Learning on IoT Devices [Lin *et al.*, NeurIPS 2020]

Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

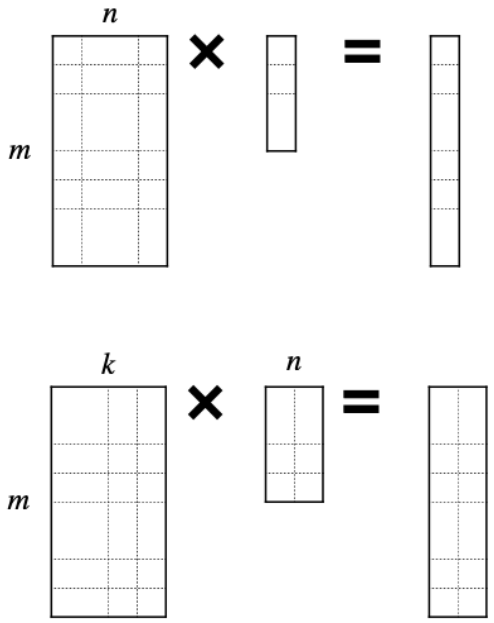
FLOP, FLOPS

OP, OPS

Number of Multiply-Accumulate Operations

MAC

- Multiply-Accumulate operation (MAC)
  - $a \leftarrow a + b \cdot c$
- Matrix-Vector Multiplication (MV)
  - $MACs = m \cdot n$
- General Matrix-Matrix Multiplication (GEMM)
  - $MACs = m \cdot n \cdot k$

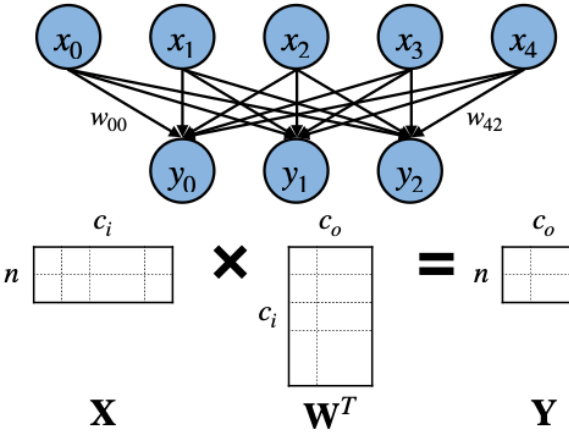


Number of Multiply-Accumulate Operations

MAC

Layer	MACs (batch size n=1)
Linear Layer	$c_o \cdot c_i$
Convolution	
Grouped Convolution	
Depthwise Convolution	

* bias is ignored



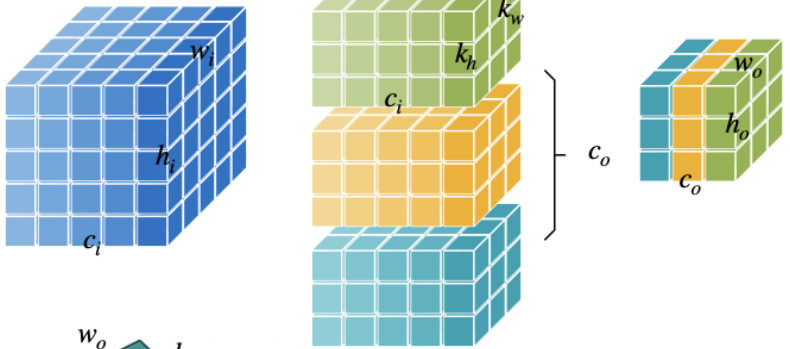
Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h, k_w	Kernel Height/Width
g	Groups

Number of Multiply-Accumulate Operations

MAC

Layer	MACs (batch size n=1)
Linear Layer	$c_o \cdot c_i$
Convolution	$c_i \cdot k_h \cdot k_w \cdot h_o \cdot w_o \cdot c_o$
Grouped Convolution	
Depthwise Convolution	

* bias is ignored



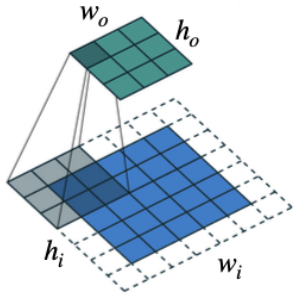


Image source: 1

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h, k_w	Kernel Height/Width
g	Groups

AlexNet: #MACs



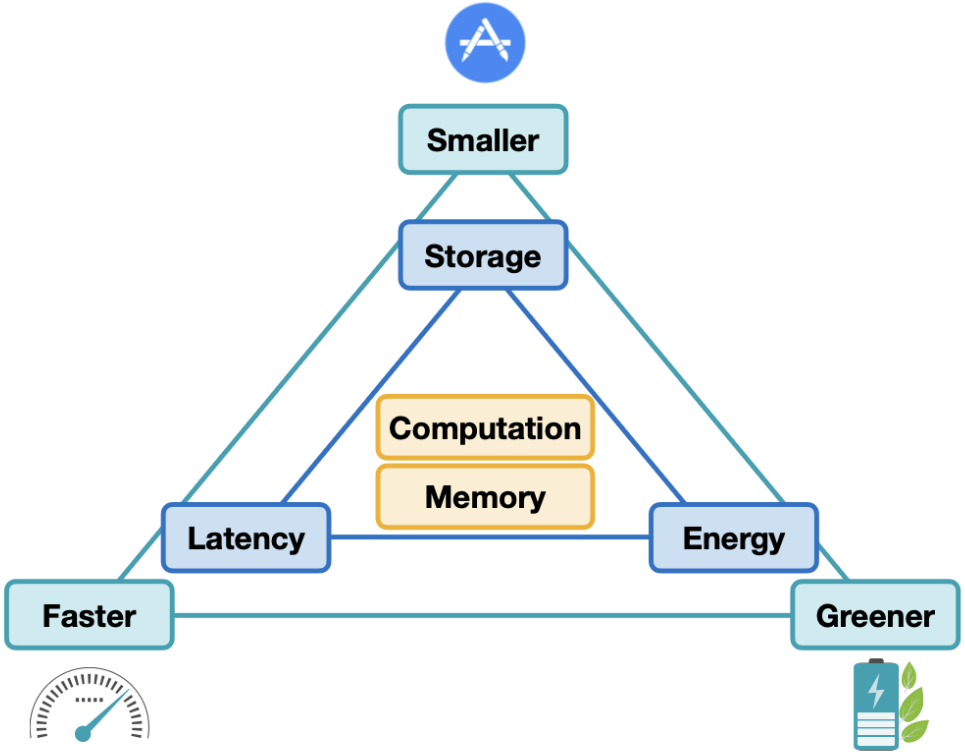
AlexNet	C × H × W	MACs
Image (3×224×224)	3×224×224	
11×11 Conv, channel 96, stride 4, pad 2	96×55×55	96×3×11×11×55×55 = 105,415,200
3×3 MaxPool, stride 2	96×27×27	
5×5 Conv, channel 256, pad 2, groups 2	256×27×27	256×96×5×5×27×27 / 2 = 223,948,800
3×3 MaxPool, stride 2	256×13×13	
3×3 Conv, channel 384, pad 1	384×13×13	384×256×3×3×13×13 = 149,520,384
3×3 Conv, channel 384, pad 1, groups 2	384×13×13	384×384×3×3×13×13 / 2 = 112,140,288
3×3 Conv, channel 256, pad 1, groups 2	256×13×13	256×[illegible]×3×13×13 / 2 = 74,[illegible]
3×3 MaxPool, stride 2	256×6×6	
Linear, channel 4096	4096	4096×(256×6×6) = 37,748,736
Linear, channel 4096	4096	4096×4096 = 16,777,216
Linear, channel 1000	1000	1000×4096 = 4,096,000

Layer	MACs (batch size n=1)
Linear Layer	$c_o \cdot c_i$
Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w \cdot h_o \cdot w_o$
Grouped Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w \cdot h_o \cdot w_o / g$
Depthwise Convolution	$c_o \cdot k_h \cdot k_w \cdot h_o \cdot w_o$

724M in total !

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky *et al.*, NeurIPS 2012]

Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

FLOP, FLOPS

OP, OPS

Number of Floating-Point Operations (FLOP)



- A multiply is a floating point operation
- An add is a floating point operation
- ***One*** multiply-accumulate operation is ***two*** floating point operations (FLOP)
  - Example: AlexNet has 724M MACs, total number of floating point operations will be
    - 724M × 2 = 1.4G FLOPs

- Floating Point Operation Per Second (FLOPS)
  - $FLOPS = \frac{FLOPs}{second}$

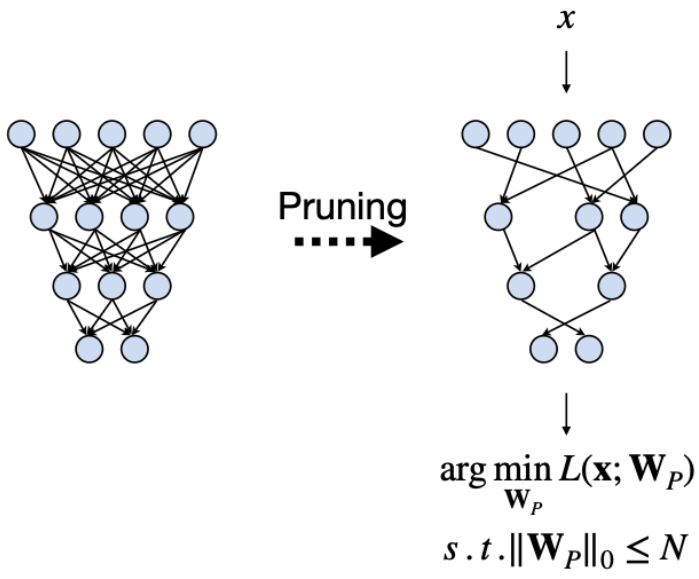
Number of Operations (OP)

- Activations/weights in neural network computing are not always floating point.
- To generalize, number of operations is used to measure the computation amount.
  - Example: AlexNet has 724M MACs, total number of operations will be
    - 724M × 2 = 1.4G OPs

- Similarly, Operation Per Second (OPS)
  - $$OPS = \frac{OPs}{second}$$

Neural Network Pruning

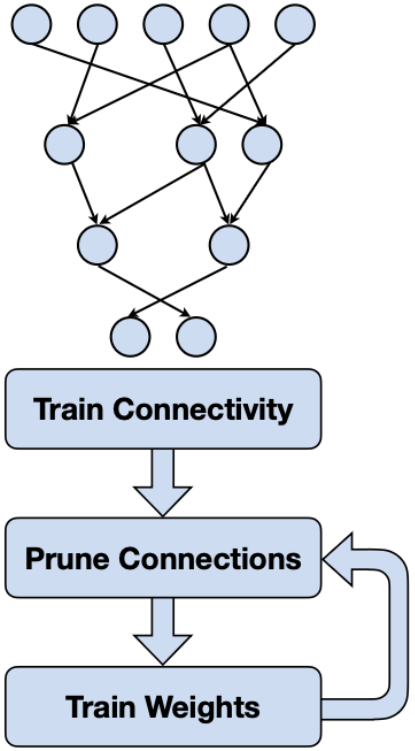
- **Introduction to Pruning**
  - What is pruning?
  - How should we formulate pruning?
- **Determine the Pruning Granularity**
  - In what pattern should we prune the neural network?
- **Determine the Pruning Criterion**
  - What synapses/neurons should we prune?
- **Determine the Pruning Ratio**
  - What should target sparsity be for each layer?
- **Fine-tune/Train Pruned Neural Network**
  - How should we improve performance of pruned models?

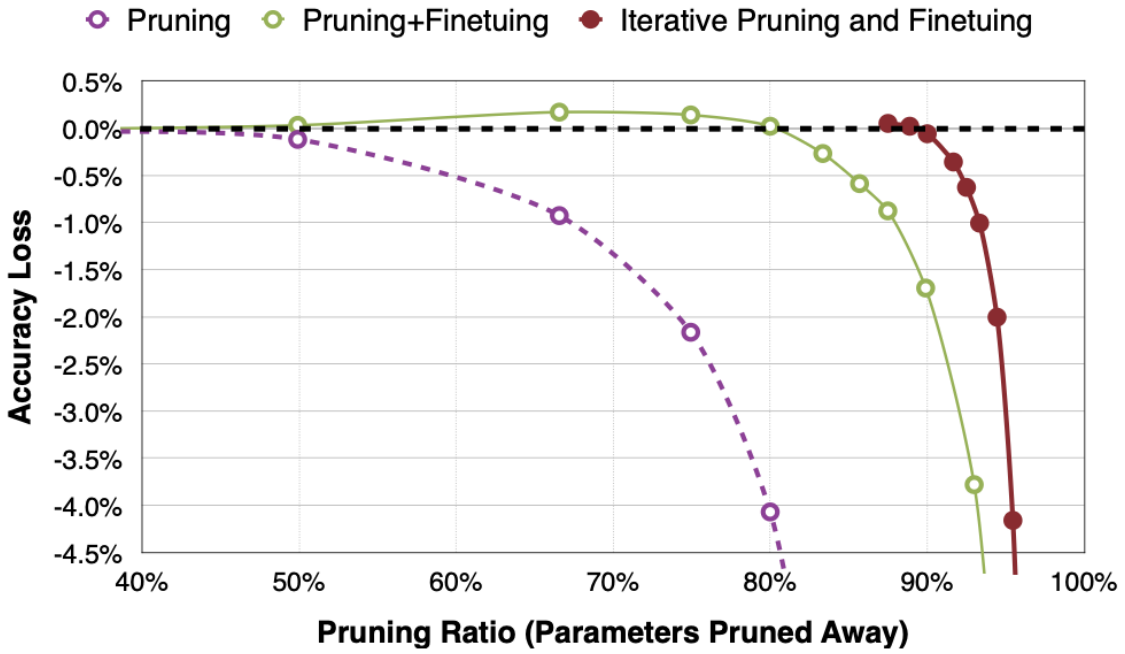


Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Neural Network Pruning

Make neural network smaller by removing parameters and activations





Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

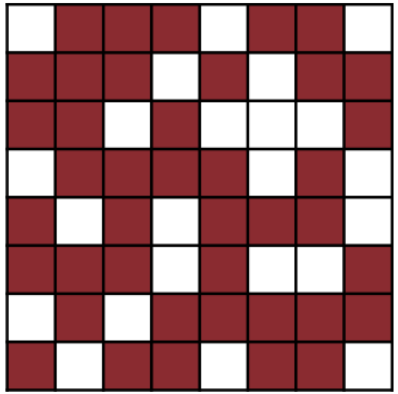
Pruning Granularity

Pruning can be performed at different granularities, from structured to non-structured.



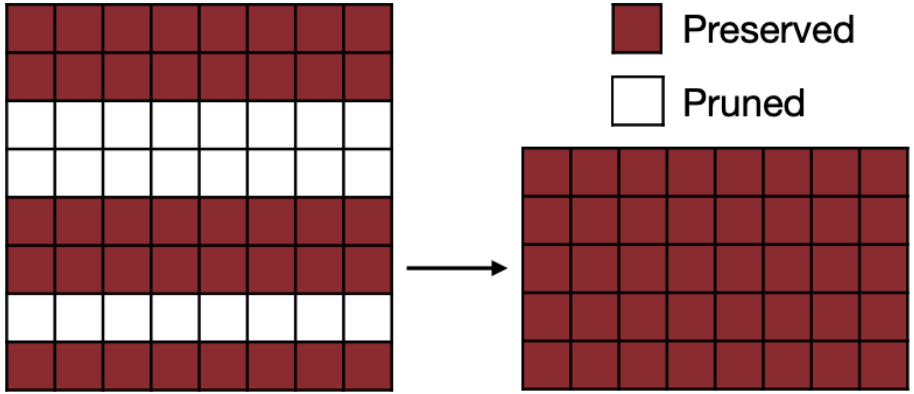
Pruning at Different Granularities

A simple example of 2D weight matrix



Fine-grained/Unstructured

- More flexible pruning index choice
- Hard to accelerate (irregular)



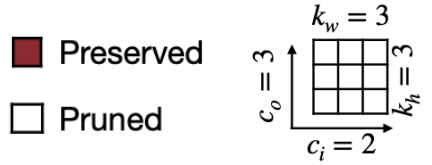
Coarse-grained/Structured

- Less flexible pruning index choice (a subset of the fine-grained case)
- Easy to accelerate (just a smaller matrix!)

Pruning at Different Granularities

The case of convolutional layers

- Some of the commonly used pruning granularities

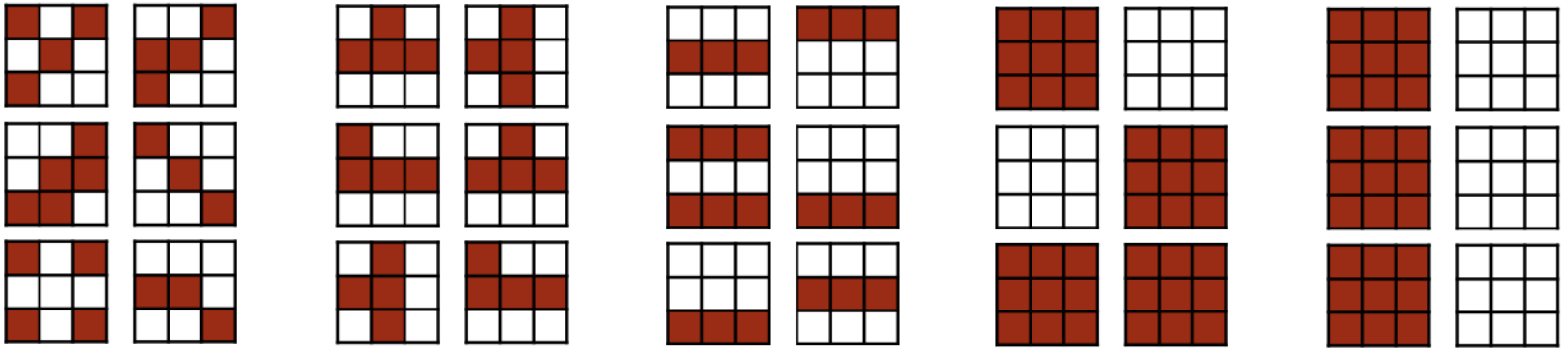


Pros?

Cons?

Irregular

Regular



Fine-grained Pruning

Pattern-based Pruning

Vector-level Pruning

Kernel-level Pruning

Channel-level Pruning

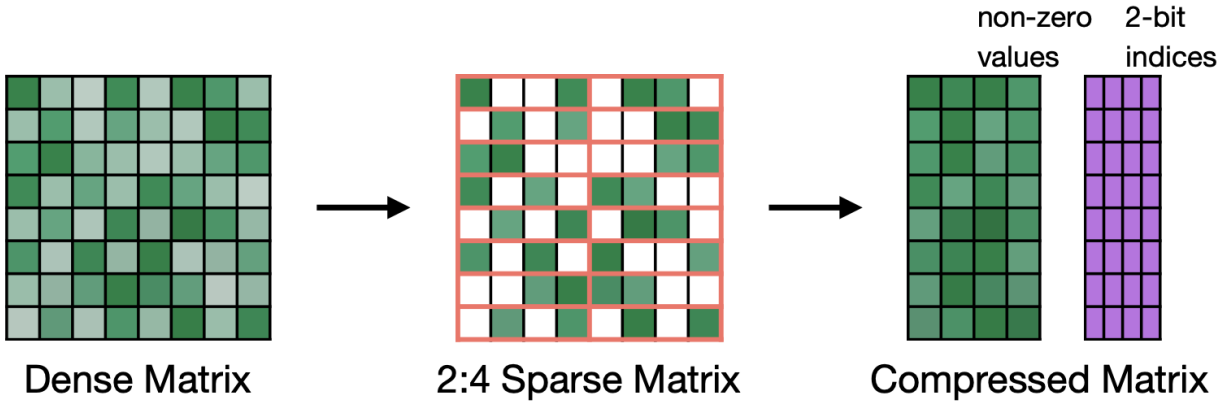
like Tetris :)

Exploring the granularity of sparsity in convolutional neural networks [Mao *et al.*, CVPR-W]

Pruning at Different Granularities

Let's look into some cases

- **Pattern-based Pruning: N:M sparsity**
  - N:M sparsity means that in each contiguous M elements, N of them is pruned
  - A classic case is 2:4 sparsity (50% sparsity)
  - It is supported by NVIDIA's Ampere GPU Architecture, which delivers up to 2x speed up



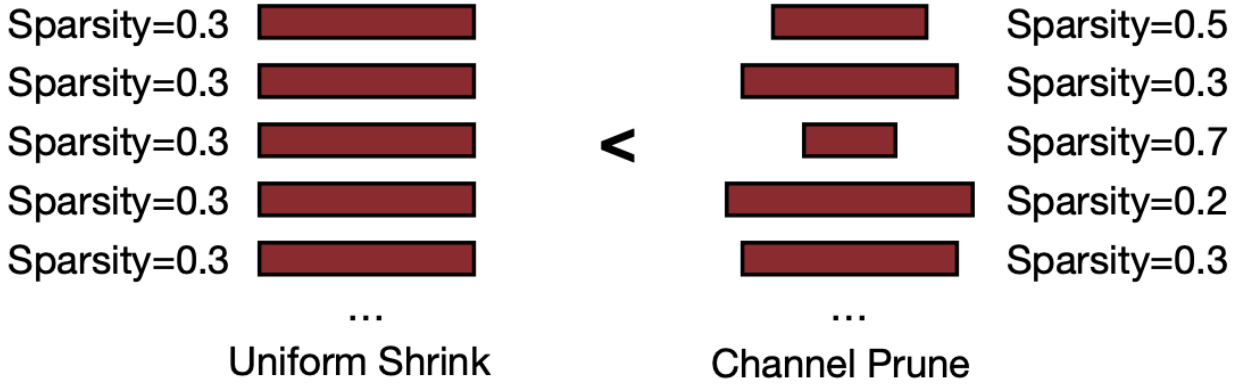
Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT

Pruning at Different Granularities

Let's look into some cases

- **Channel Pruning**
  - Pro: Direct speed up due to reduced channel numbers (leading to an NN with smaller #channels)
  - Con: smaller compression ratio

We will later discuss how to find sparsity ratios



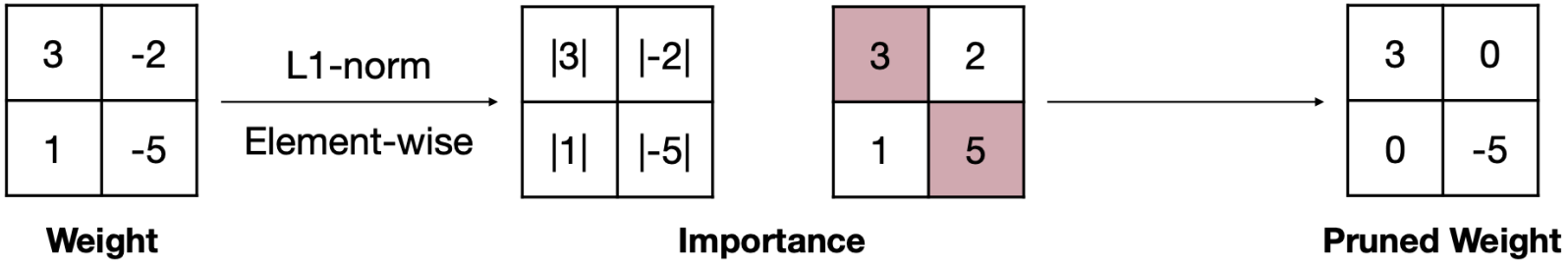
Magnitude-based Pruning

A heuristic pruning criterion

- Magnitude-based pruning considers weights with ***larger absolute values*** are more important than other weights.
  - For element-wise pruning,

$$Importance = |W|$$

- **Example**

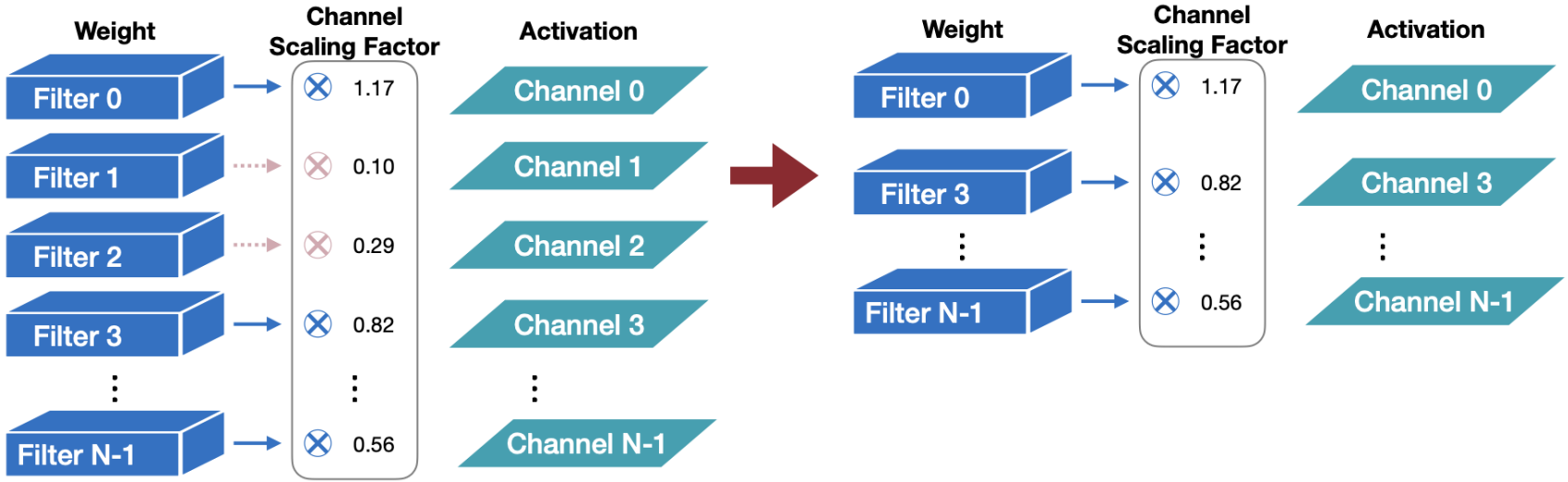


Learning Both Weights and Connections for Efficient Neural Network [Han *et al*., NeurIPS 2015]

Scaling-based Pruning

Pruning criterion for filter pruning

- A scaling factor is associated with each filter (*i.e.*, output channel) in convolutional layers
  - The scaling factor is multiplied to the output of that channel
  - The scaling factors are trainable parameters
- The filters/output channels with small scaling factor magnitude will be pruned



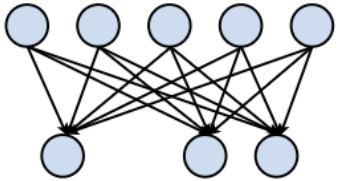
Learning Efficient Convolutional Networks through Network Slimming [Liu *et al.*, ICCV 2017]

Selection of Neurons (Activations) to Prune

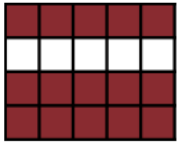
- When removing neurons from a neural network model,
  - ***the less useful*** the neurons being removed are,
  - the better the performance of pruned neural network is.

Neuron pruning is coarse-grained weight pruning

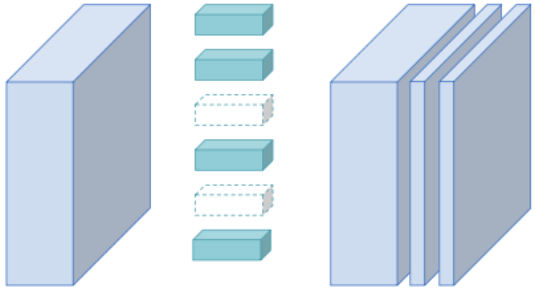
Neuron Pruning in Linear Layer

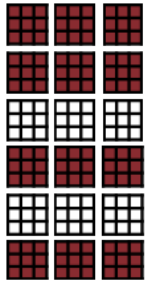


Weight Matrix



Channel Pruning in Convolution Layer

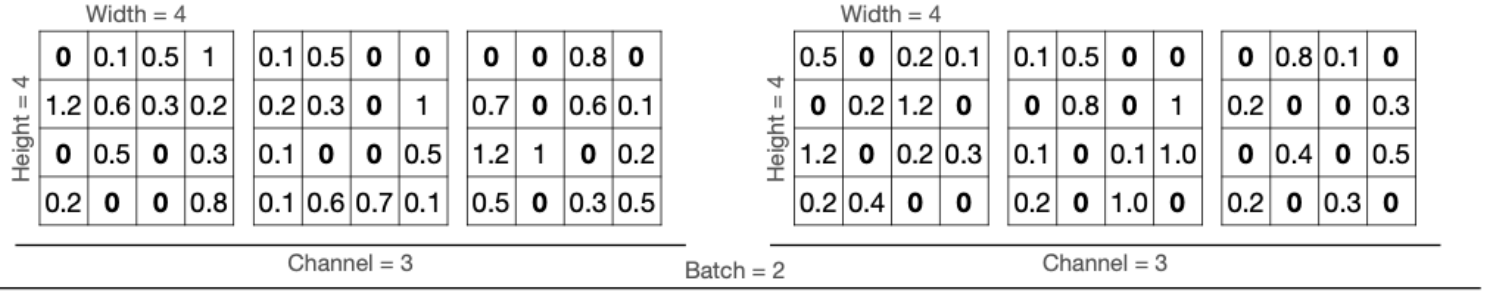




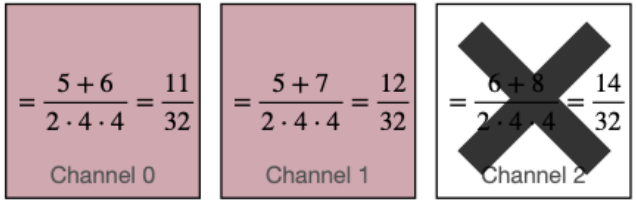
Percentage-of-Zero-Based Pruning

- ReLU activation will generate zeros in the output activation.
- Similar to magnitude of weights, the Average Percentage of Zero activations (APoZ) can be exploited to measure the importance of the neurons.
- The smaller APoZ is, the more importance the neuron has.

Output Activations



Average Percentage of Zeros (APoZ)



Network Trimming: A Data-Driven Neuron Pruning Approach towards Efficient Deep Architectures [Hu *et al.*, ArXiv 2017]

Finding Pruning Ratios



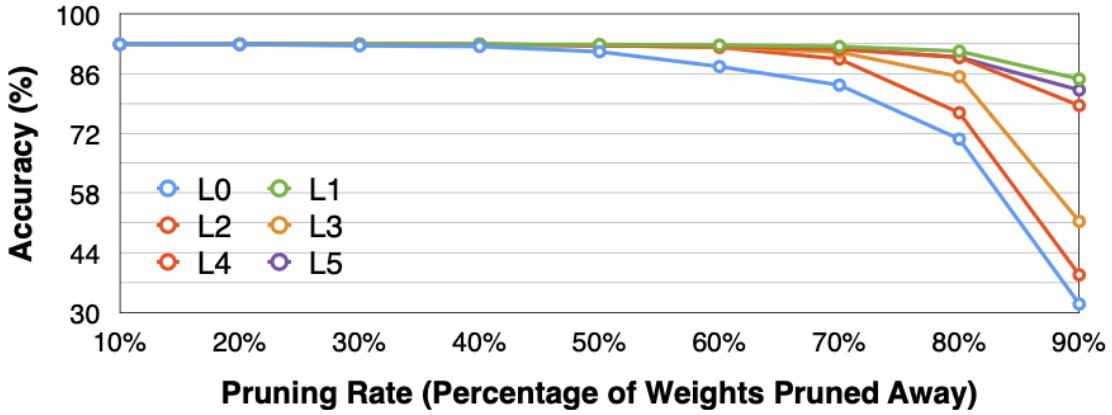
Analyze the sensitivity of each layer

- We need different pruning ratios for each layer since different layers have different **sensitivity**
  - Some layers are more sensitive (e.g., first layer)
  - Some layers are more redundant
- We can perform **sensitivity analysis** to determine the per-layer pruning ratio

Finding Pruning Ratios

Analyze the sensitivity of each layer

- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, ..., 0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
  - Repeat the process for all layers

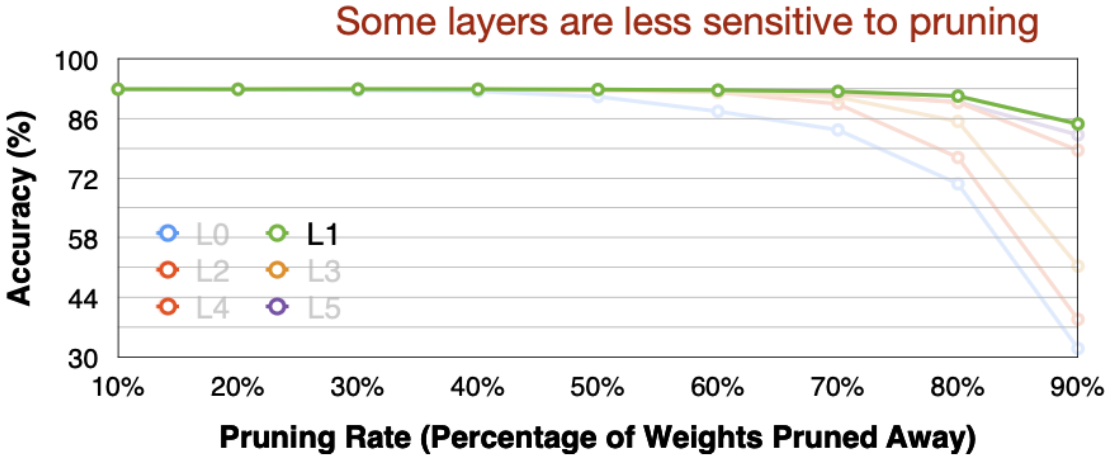


Finding Pruning Ratios



Analyze the sensitivity of each layer

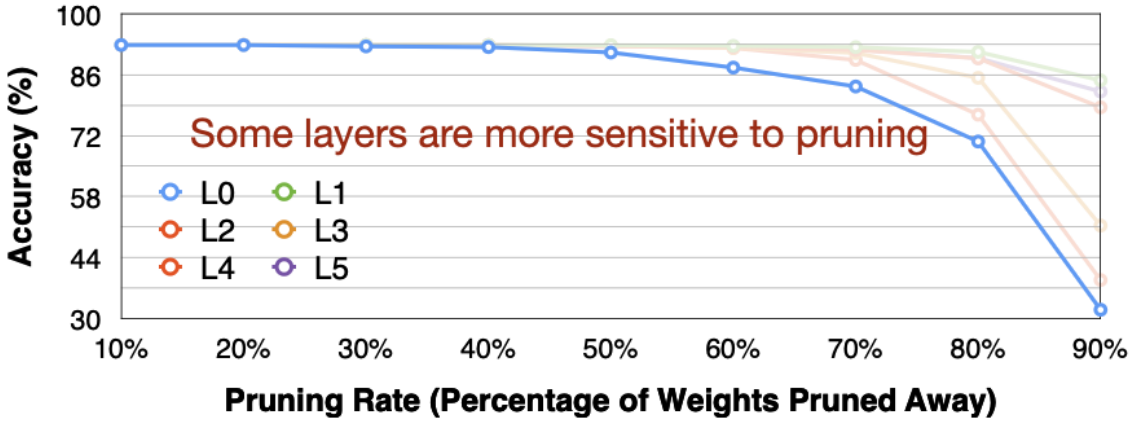
- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, ..., 0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
  - Repeat the process for all layers



Finding Pruning Ratios

Analyze the sensitivity of each layer

- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, ..., 0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
  - Repeat the process for all layers

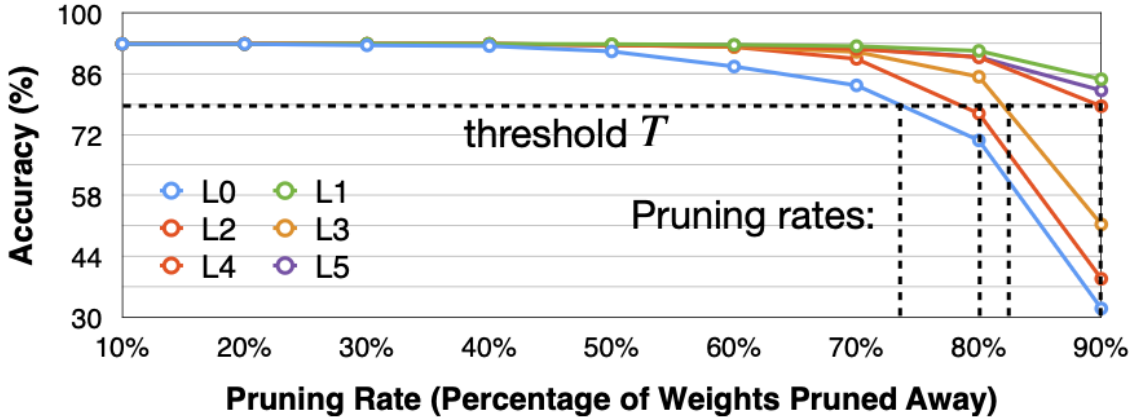


Finding Pruning Ratios



Analyze the sensitivity of each layer

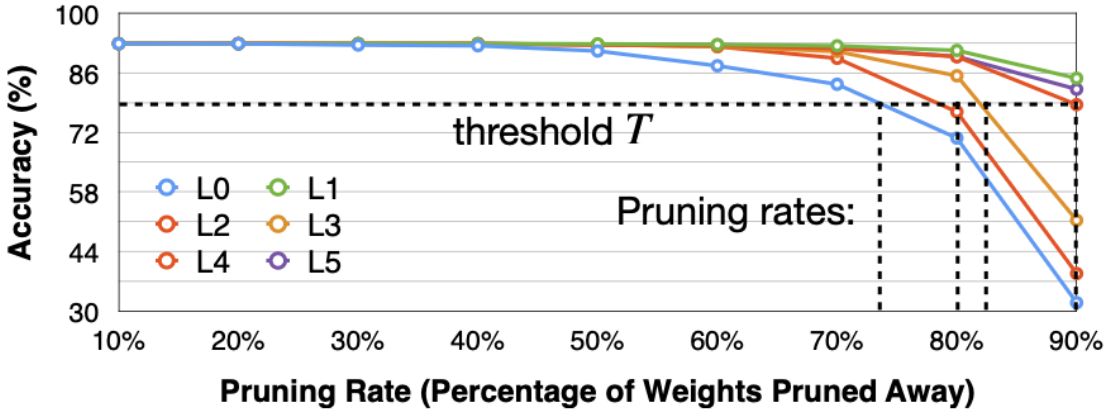
- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, ..., 0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
  - Repeat the process for all layers
  - Pick a degradation threshold T such that the overall pruning rate is desired



Finding Pruning Ratios

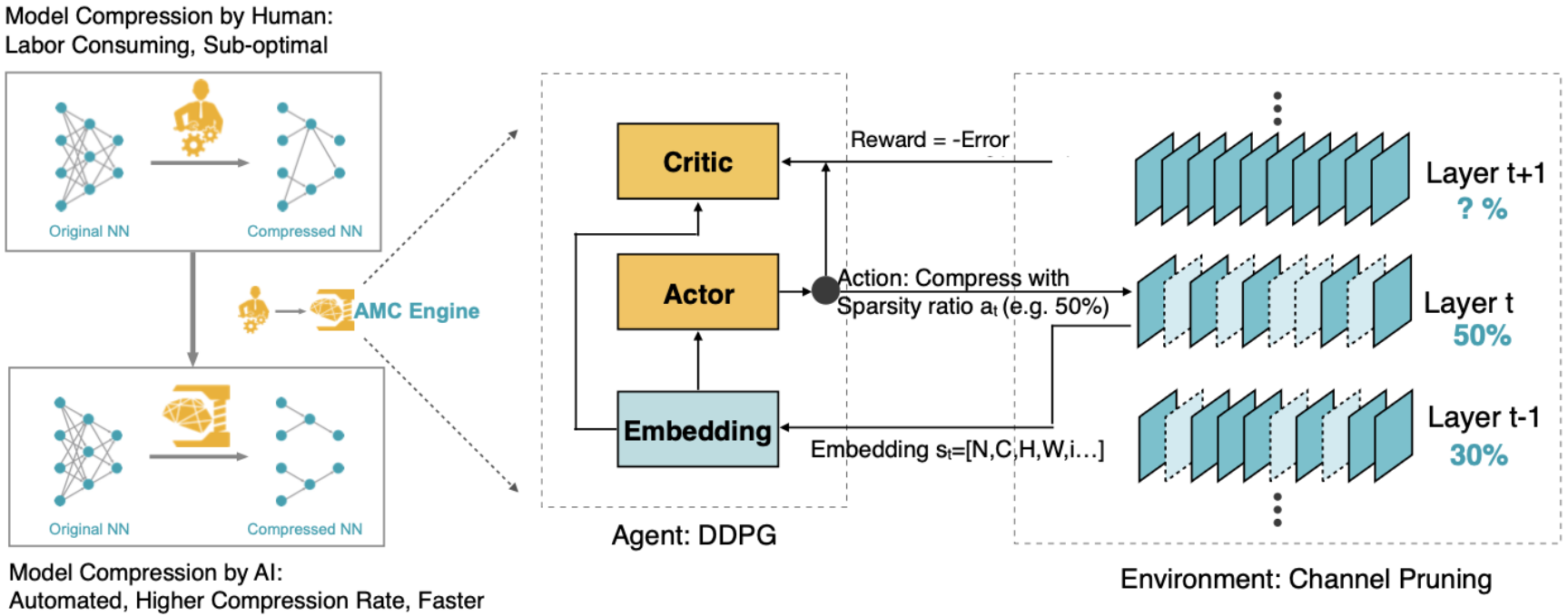
Analyze the sensitivity of each layer

- Is this optimal?
  - Maybe not. We do not consider the interaction between layers.
- Can we go beyond the heuristics?
  - Yes!



AMC: AutoML for Model Compression

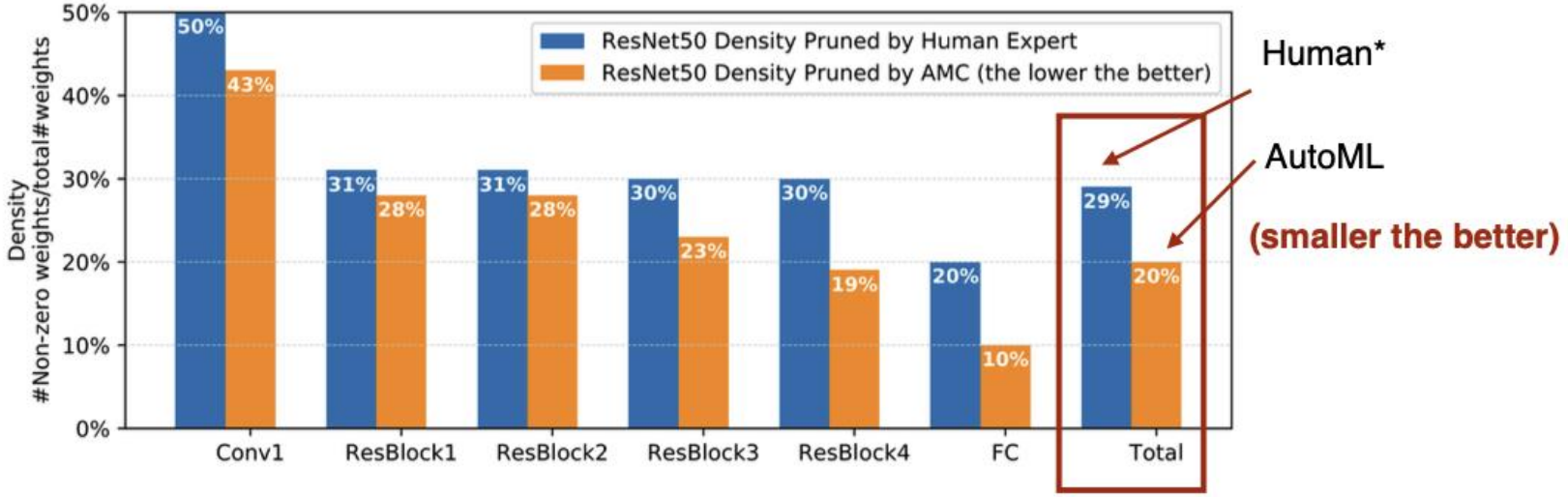
Pruning as a reinforcement learning problem



AMC: AutoML for Model Compression and Acceleration on Mobile Devices [He *et al.*, ECCV 2018]

AMC: AutoML for Model Compression

Pruning as a reinforcement learning problem

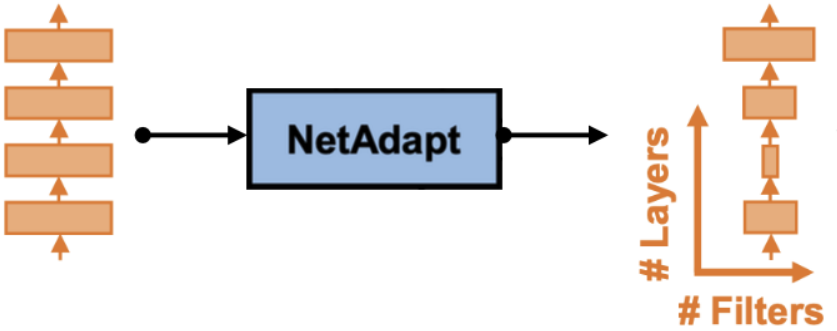


* Efficient Methods and Hardware for Deep Learning [Han, *thesis*]

NetAdapt

A rule-based iterative/progressive method

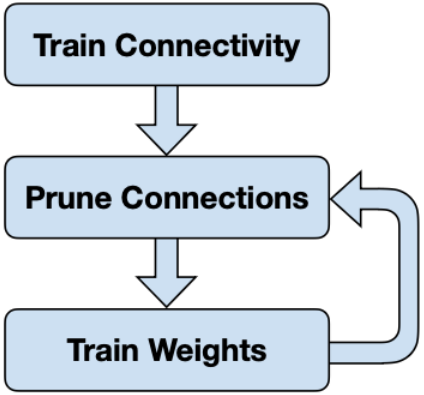
- The goal of NetAdapt is to find a per-layer pruning ratio to meet a global resource constraint (e.g., latency, energy, …)
- The process is done iteratively
- We take **latency** constraint as an example

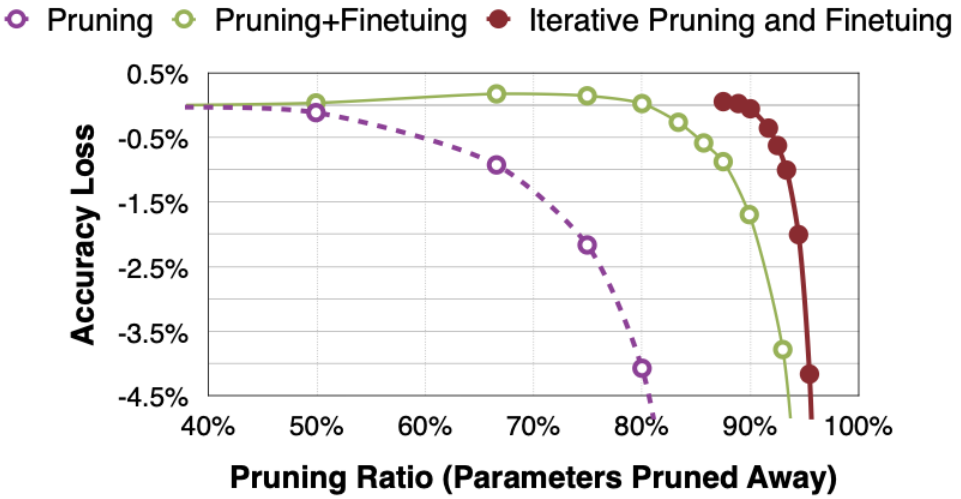


NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

Iterative Pruning

- Consider pruning followed by a fine-tuning is one iteration.
- Iterative pruning gradually increases the target sparsity in each iteration.
  - boost pruning ratio from 5X to 9X on AlexNet compared to single-step aggressive pruning.





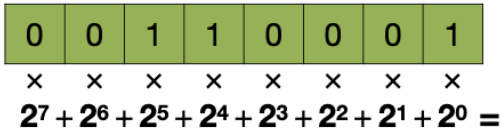
Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Integer

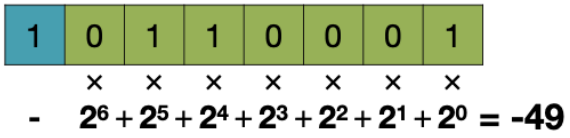
- Unsigned Integer
  - *n*-bit Range: $[0,\ 2^n - 1]$

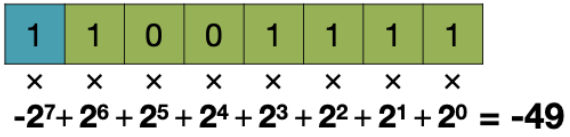
- Signed Integer
  - Sign-Magnitude Representation
    - *n*-bit Range: $[-2^{n-1} + 1,\ 2^{n-1} - 1]$
    - Both 000…00 and 100…00 represent 0

  - Two's Complement Representation
    - *n*-bit Range: $[-2^{n-1},\ 2^{n-1} - 1]$
    - 000…00 represents 0
    - 100…00 represents -2^{n-1}



Sign Bit





Integer



- 11000000
- Calculate its decimal value under
  - Unsigned:

    $11000000 = 1\times2^7+1\times2^6+0\times2^5+\ldots+0\times2^0 = 192$
  - Signed:

    $11000000 = -1\times2^6+0\times2^5+\ldots+0\times2^0 = -64$
  - Two's complement:

    $11000000 = -1\times2^7+1\times2^6+0\times2^5+\ldots+0\times2^0 = -64$

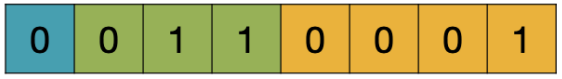


Fixed-Point Number
(two's complement representation)

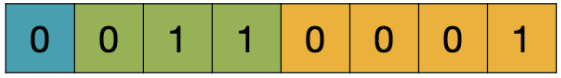


Integer . Fraction

"Decimal" Point



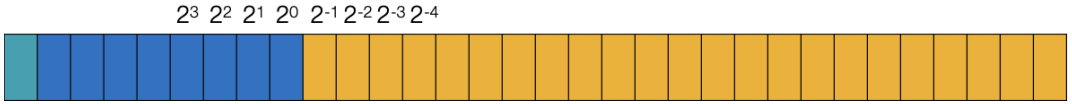
$$-2^3+2^2+2^1+2^0+2^{-1}+2^{-2}+2^{-3}+2^{-4} = 3.0625$$



$$\left(-2^7+2^6+2^5+2^4+2^3+2^2+2^1+2^0\right) \times 2^{-4} = 49 \times 0.0625 = 3.0625$$

Floating-Point Number

Example: 32-bit floating-point number in IEEE 754

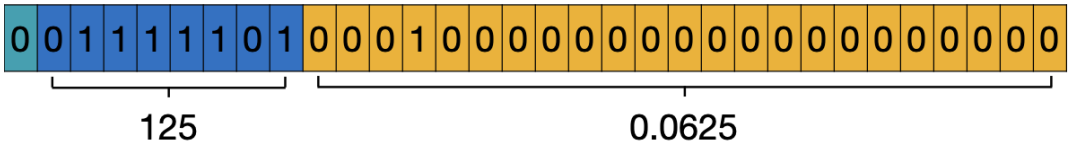


Sign **8 bit Exponent** **23 bit Fraction** (significant / mantissa)

$$(-1)^{\text{sign}} \times (1 + \textbf{Fraction}) \times 2^{\textbf{Exponent}-127}$$ ← Exponent Bias = 127 = $2^{8-1}-1$

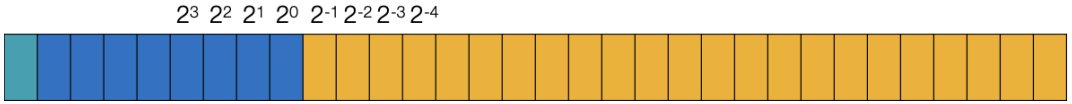
How to represent **0.265625**?

0.265625 = $1.0625 \times 2^{-2} = (1 + \underline{0.0625}) \times 2^{\underline{125}-127}$



Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



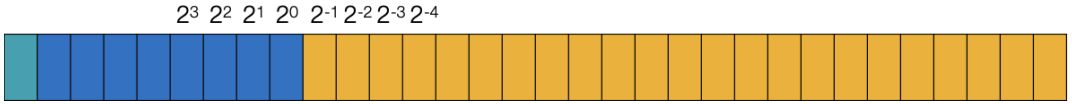
Sign **8 bit Exponent** **23 bit Fraction** (significant / mantissa)

$$(-1)^{\text{sign}} \times (1 + \textbf{Fraction}) \times 2^{\textbf{Exponent}-127}$$ ← Exponent Bias = 127 = $2^{8-1}-1$

The exponent bias allows both positive and negative exponents to be stored using only nonnegative bit patterns, making the hardware simpler and the data consistent.

Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



Sign **8 bit Exponent** **23 bit Fraction** (significant / mantissa)

$$(-1)^{sign} \times (1 + \textbf{Fraction}) \times 2^{\textbf{Exponent}-127}$$ ← Exponent Bias = 127 = $2^{8-1}-1$

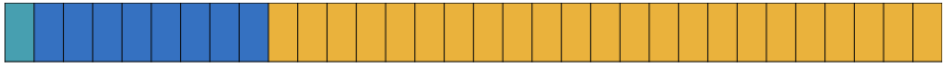
How should we represent 0?



125 0.0625

Floating-Point Number

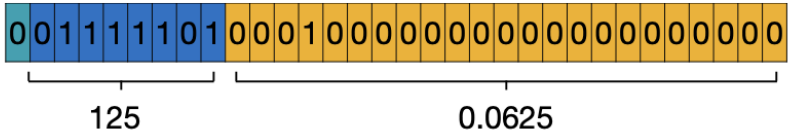
Example: 32-bit floating-point number in IEEE 754



Sign **8 bit Exponent** **23 bit Fraction**

$(-1)^{sign} \times (1 + \textbf{Fraction}) \times 2^{\textbf{Exponent}-127}$

Normal Numbers, Exponent≠0

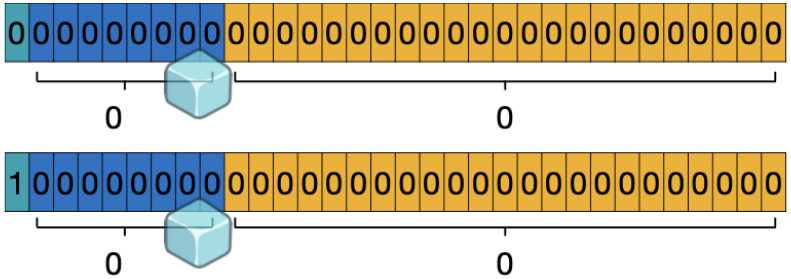


$0.265625 = 1.0625 \times 2^{-2} = (1 + \underline{0.0625}) \times 2^{\underline{125}-127}$

Should have been $(-1)^{sign} \times (1 + \textbf{Fraction}) \times 2^{0-127}$

But we force to be $(-1)^{sign} \times \textbf{Fraction} \times 2^{1-127}$

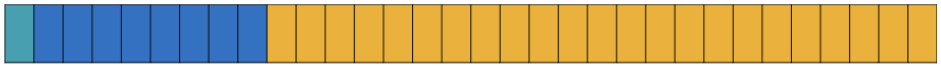
Subnormal Numbers, Exponent=0



$0 = 0 \times 2^{-126}$

Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



Sign **8 bit Exponent** **23 bit Fraction**

$(-1)^{sign} \times (1 + \textbf{Fraction}) \times 2^{\textbf{Exponent}-127}$

Normal Numbers, Exponent≠0

$(-1)^{sign} \times \textbf{Fraction} \times 2^{1-127}$

Subnormal Numbers, Exponent=0

What is the largest positive subnormal value?

Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



Sign 8 bit Exponent **23 bit Fraction**

$(-1)^{sign} \times (1 + \textbf{Fraction}) \times 2^{\textbf{Exponent}-127}$

Normal Numbers, Exponent≠0


0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0

Floating-Point Number

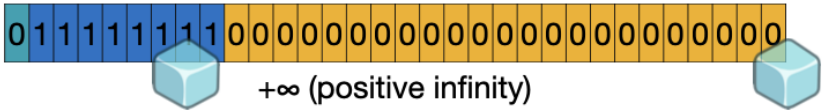
Example: 32-bit floating-point number in IEEE 754

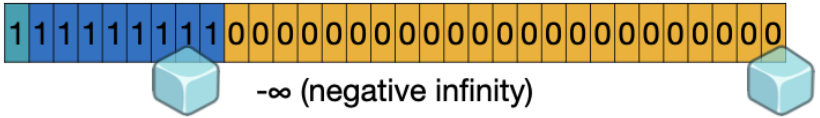


Sign 8 bit Exponent **23 bit Fraction**

$(-1)^{sign} \times (1 + \textbf{Fraction}) \times 2^{\textbf{Exponent}-127}$

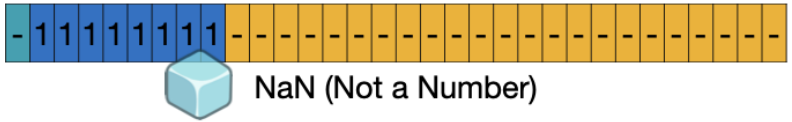
Normal Numbers, Exponent≠0





$(-1)^{sign} \times \textbf{Fraction} \times 2^{1-127}$

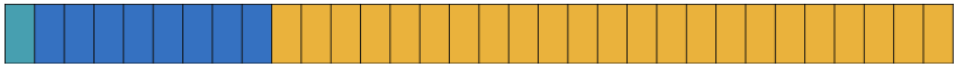
Subnormal Numbers, Exponent=0



much waste. revisit in fp8.

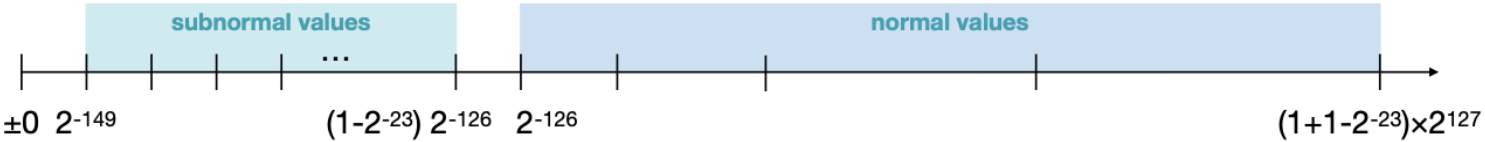
Floating-Point Number

Example: 32-bit floating-point number in IEEE 754



Sign 8 bit Exponent **23 bit Fraction**

Exponent	Fraction=0	Fraction≠0	Equation
$00_H = 0$	±0	subnormal	$(-1)^{sign} \times \mathbf{Fraction} \times 2^{1-127}$
$01_H \ldots FE_H = 1 \ldots 254$	normal		$(-1)^{sign} \times (1 + \mathbf{Fraction}) \times 2^{\mathbf{Exponent}-127}$
$FF_H = 255$	±INF	NaN	



Floating-Point Number

Exponent Width -> Range; Fraction Width -> Precision

IEEE 754 Single Precision 32-bit Float (IEEE FP32)



IEEE 754 Half Precision 16-bit Float (IEEE FP16)



Google Brain Float (BF16)



Exponent (bits)	Fraction (bits)	Total (bits)
8	23	32
5	10	16
8	7	16

Numeric Data Types

- **Question**: What is the following IEEE half precision (IEEE FP16) number in decimal?



Exponent Bias = 15_{10}

  - Sign: -
  - Exponent: $10001_2 - 15_{10} = 17_{10} - 15_{10} = 2_{10}$
  - Fraction: $1100000000_2 = 0.75_{10}$
  - Decimal Answer = $-(1 + 0.75) \times 2^2 = -1.75 \times 2^2 = -7.0_{10}$

Floating-Point Number

Exponent Width -> Range; Fraction Width -> Precision

	Exponent (bits)	Fraction (bits)	Total (bits)
IEEE 754 Single Precision 32-bit Float (IEEE FP32) 	8	23	32
IEEE 754 Half Precision 16-bit Float (IEEE FP16) 	5	10	16
Nvidia FP8 (E4M3)  * FP8 E4M3 does not have INF, and S.1111.111$_2$ is used for NaN. * Largest FP8 E4M3 normal value is S.1111.110$_2$ =448.	4	3	8
Nvidia FP8 (E5M2) for gradient in the backward  * FP8 E5M2 have INF (S.11111.00$_2$) and NaN (S.11111.XX$_2$). * Largest FP8 E5M2 normal value is S.11110.11$_2$ =57344.	5	2	8

Floating-Point Number

Exponent Width -> Range; Fraction Width -> Precision



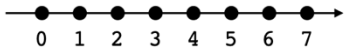
INT4

S			
0	0	0	1
0	1	1	1

=1

=7

-1,-2,-3,-4,-5,-6,-7,-8
0, 1, 2, 3, 4, 5, 6, 7



-1,-2,-3,-4,-5,-6,-7,-8
0, 1, 2, 3, 4, 5, 6, 7

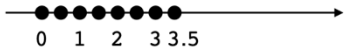
FP4 (E1M2)

S	E	M	M
0	0	0	1
0	1	1	1

$=0.25\times2^{1-0}=0.5$

$=(1+0.75)\times2^{1-0}=3.5$

-0,-0.5,-1,-1.5,-2,-2.5,-3,-3.5
0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5



-0,-1,-2,-3,-4,-5,-6,-7 ×0.5
0, 1, 2, 3, 4, 5, 6, 7

FP4 (E2M1)

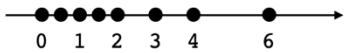
S	E	E	M
0	0	0	1
0	1	1	1

$=0.5\times2^{1-1}=0.5$

$=(1+0.5)\times2^{3-1}=1$

no inf, no NaN

-0,-0.5,-1,-1.5,-2,-3,-4,-6
0, 0.5, 1, 1.5, 2, 3, 4, 6



-0,-1,-2,-3,-4,-6,-8,-12 ×0.5
0, 1, 2, 3, 4, 6, 8, 12

FP4 (E3M0)

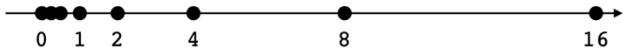
S	E	E	E
0	0	0	1
0	1	1	1

$=(1+0)\times2^{1-3}=0.25$

$=(1+0)\times2^{7-3}=16$

no inf, no NaN

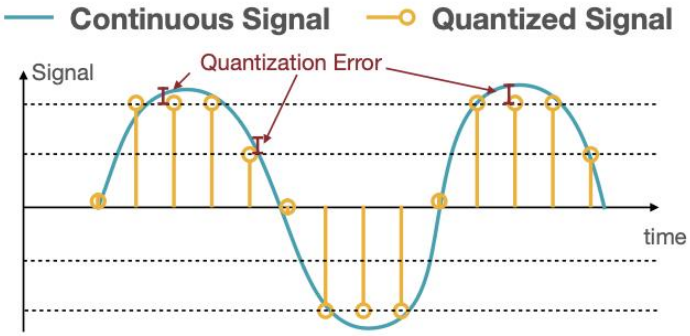
-0,-0.25,-0.5,-1,-2,-4,-8,-16
0, 0.25, 0.5, 1, 2, 4, 8, 16



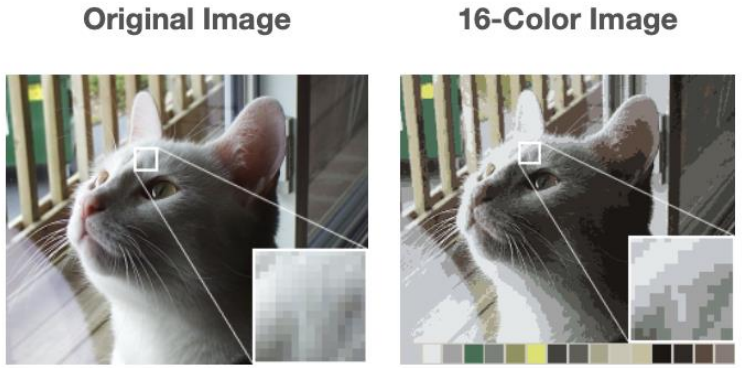
-0,-1,-2,-4,-8,-16,-32,-64 ×0.25
0, 1, 2, 4, 8, 16, 32, 64

What is Quantization?

Quantization is the process of constraining an input from a continuous or otherwise large set of values to a discrete set.



The difference between an input value and its quantized value is referred to as quantization error.



Images are in the public domain.

"Palettization"